MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE RUSSIAN FEDERATION

Federal State Autonomous Educational Institution of Higher Education
"Peter the Great St. Petersburg Polytechnic University"

Institute of Computer Science and Cybersecurity

Higher School of Technology and Artificial Intelligence

Program: 02.03.01 Mathematics and Computer Science

Coursework report
on the discipline "Graph Theory"
Implementation of graph theory algorithms
for laboratory works No. 1–5

Student,
group 5130201/40001 _____ **Ovi Md Shamin Yasir**

Teacher,

_____ **Vostrov Alexey Vladimirovich**

"____"_____ **2026**

St. Petersburg, 2026

Contents

Introduction

This report describes 5 laboratory works. Each work includes the implementation of different algorithms on randomly generated connected acyclic graphs. The graphs are generated using the *normal distribution* according to Vadzinsky's handbook [7].

LABORATORY WORK No. 1

1. Randomly generate a connected acyclic directed graph according to the *normal distribution* with a user-defined number of vertices. The distribution parameters are constants selected experimentally. The distribution is taken from Vadzinsky's handbook [7]. Generation is performed by vertex degrees.
2. Calculate vertex eccentricities, determine the center of the graph and the diametral vertices.
3. Implement Shimbell's method for the weight matrix generated by the specified distribution. The user enters the number of edges. The answer must contain the minimum and/or maximum path represented as a matrix. The program must support generating only positive, only negative, and mixed weights at the user's choice.
4. Determine whether a route can be built from one specified vertex to another, where both vertices are entered by the user, and output the number of such routes.

Variant: normal distribution.

LABORATORY WORK No. 2

1. For the given graphs randomly generated in the previous work, perform *depth-first graph traversal.*
2. Generate a weight matrix, either positive or containing negative weights at the user's choice, and find the shortest path between two user-selected vertices using the *Floyd-Warshall algorithm.* The result includes the distance matrix and the path itself as a sequence of vertices.
3. Compare the running speed of the implemented algorithms by the number of iterations.

Variant: (c) depth-first graph traversal; (j) Floyd-Warshall algorithm.

LABORATORY WORK No. 3

1. Based on the generated directed graph, randomly obtain the capacity and cost matrices.
2. For the obtained graph, find the maximum flow using the *Ford-Fulkerson algorithm.*
3. Compute the specified minimum-cost flow, where the required flow value is $\lfloor 2/3 \cdot \max \rfloor$ and max is the maximum flow. Use the previously implemented Floyd-Warshall algorithm.

LABORATORY WORK No. 4

1. For the given undirected graphs randomly generated in the first work, find the number of spanning trees using Kirchhoff's matrix-tree theorem.
2. Build a minimum-weight spanning tree for the generated undirected weighted graph using *Boruvka's algorithm.* Encode the obtained spanning tree with the Prufer code and decode it while preserving the edge weights.
3. Implement *minimum edge cover* construction on the original graph and on the obtained spanning tree at the user's choice.

Variant: (b) Boruvka's algorithm; (g) minimum edge cover.

LABORATORY WORK No. 5

1. For the given undirected graphs randomly generated in the first work, check whether the graph is Eulerian. If it is not, modify the graph and log what was changed. Build an Eulerian cycle.
2. Build the shortest spanning tree using Boruvka's algorithm from the fourth work. Based on this spanning tree, obtain the *fundamental cutset system* and print it. Implement obtaining all other cutsets from the fundamental ones using the symmetric difference operation; the selected cutsets are entered by the user.

Variant: 36, even – fundamental cutset system.

The laboratory works were implemented in C++ in the CLion development environment.

1 Mathematical Description

1.1 Graph

A simple graph $G(V, E)$ is a combination of two sets: a non-empty set V and a set E of unordered pairs of different elements of V. The set V is called the set of vertices, and the set E is called the set of edges.

$$G(V, E) = \langle V, E \rangle, \quad V \neq \emptyset, \quad E \subseteq V \times V, \quad \{v, v\} \notin E, \quad v \in V,$$

that is, the set E consists of 2-element subsets of the set V.

Related terms:

- A vertex of a graph G is an element of the vertex set $v \in V(G)$;
- An edge of a graph G is an element of the edge set $e \in E(G)$, or $e = \{v_1, v_2\}$, where $v_1 \in V(G)$ and $v_2 \in V(G)$;
- The elements of a graph G are its vertices $v \in V(G)$ and edges $e \in E(G)$;
- A vertex v is incident to an edge e if $v \in e$; in this case, e is also said to be an edge at v;
- Adjacent vertices are vertices v_1 and v_2 such that $\{v_1, v_2\} \in E(G)$;
- The degree of a vertex $d(v)$ is the number of edges incident to it.

1.2 Normal Distribution

The normal, or Gaussian, distribution is a continuous probability distribution. Its probability density is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right),$$

where μ is the expected value and σ^2 is the variance.

To generate random vertex degrees, an approximate method from Vadzinsky's handbook [7] is used. A random variable X with normal distribution $\mathcal{N}(\mu, \sigma^2)$ is approximated as follows:

$$X \approx \sqrt{\frac{12}{n}} \left(\sum_{i=1}^{n} r_i - \frac{n}{2} \right),$$

where $r_i \sim U(0, 1)$ are independent random numbers uniformly distributed on $[0, 1]$, and n is the number of summands, which is the approximation accuracy parameter. The distribution is taken from the handbook on probability distributions [7].

1.3 Eccentricity, Center and Diametral Vertices

The **eccentricity** of a vertex v in a graph G is the maximum distance from v to the other vertices:

$$\varepsilon(v) = \max_{u \in V} d(v, u),$$

where $d(v, u)$ is the length of the shortest path between vertices v and u.

The **radius** of a graph is the minimum eccentricity among all vertices:

$$\mathrm{rad}(G) = \min_{v \in V} \varepsilon(v).$$

The **diameter** of a graph is the maximum eccentricity:

$$\operatorname{diam}(G) = \max_{v \in V} \varepsilon(v).$$

The **center** of a graph is the set of vertices with minimum eccentricity equal to the radius:

$$Z(G) = \{v \in V \mid \varepsilon(v) = \operatorname{rad}(G)\}.$$

Diametral vertices are vertices whose eccentricity is equal to the diameter of the graph. Eccentricities are computed using breadth-first search (BFS) from each vertex in time $O(V(V+E))$.

1.4 Shimbell's Method

Shimbell's method makes it possible to find minimum or maximum paths between vertices that consist of a specified number of edges.

- The operation of multiplying two values a and b while raising a matrix to a power corresponds to their algebraic sum, that is:

$$a * b = b * a \;\rightarrow\; a + b = b + a, \qquad a * 0 = 0 * a = 0 \;\rightarrow\; a + 0 = 0.$$

- The operation of adding two values a and b is replaced by selecting the maximum or minimum of these values, that is:

$$a + b = b + a = \min(\max)\,\{a,\, b\}.$$

Zeros are ignored in this operation.

$$w_{ij} = \min\big(\max(w_{i1} + w_{1j}), \ldots, (w_{in} + w_{nj})\big).$$

1.5 Depth-First Search Traversal

Depth-first search is one of the basic graph traversal methods. It is often used to check connectivity, find cycles and strongly connected components, and perform topological sorting.

The general idea of the algorithm is as follows: for each unvisited vertex, all unvisited adjacent vertices must be found, and the search must be repeated for them.

1. Choose any vertex among the unvisited vertices and denote it by u;
2. Run the procedure dfs(u):
   - Mark vertex u as visited;
   - For each unvisited vertex adjacent to u, denoted by v, run dfs(v);
3. Repeat steps 1 and 2 until all vertices are visited.

The running time of the algorithm is estimated as $O(V + E)$.

1.6 Floyd-Warshall Algorithm

The Floyd-Warshall algorithm is intended for finding shortest paths between all pairs of vertices in a weighted graph, including graphs with negative edge weights, provided there are no negative cycles.

Let $d^{(k)}[i][j]$ denote the length of the shortest path from vertex i to vertex j that passes only through intermediate vertices from the set $\{1, 2, \ldots, k\}$. The recurrence relation is

$$d^{(k)}[i][j] = \min\left(d^{(k-1)}[i][j],\ d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\right).$$

The initial conditions are

$$d^{(0)}[i][j] = \begin{cases} w(i,j), & \text{if edge } (i,j) \text{ exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

As a result, $d^{(n)}[i][j]$ contains the length of the shortest path from i to j. The answer contains the distance matrix and the path itself as a sequence of vertices. The complexity of the algorithm is $O(V^3)$.

1.7 Ford-Fulkerson Algorithm

The Ford-Fulkerson algorithm solves the problem of finding the maximum flow in a network. The idea of the algorithm is as follows:

- Initially, the flow value is set to 0 for all u and v in the transport network.
- Then the flow is iteratively increased by searching for an augmenting path from the source s to the sink t.
- The process is repeated while an augmenting path can be found.

For the Edmonds-Karp implementation used in the program, where augmenting paths are found by BFS, the running time is $O(VE^2)$.

The algorithm for finding a specified minimum-cost flow can use one of the shortest-path algorithms from the source to the sink. In this work, the required total flow is routed by repeatedly finding the shortest path by cost in the residual network. The objective function is

$$\min\left(\sum_{(u,v)\in E} c(u,v)\cdot f(u,v)\right),$$

where E is the set of all edges in the graph, $c(u,v)$ is the cost of flow on edge (u,v), and $f(u,v)$ is the flow on edge (u,v). In the implemented version, the shortest path in the residual cost graph is found using Floyd-Warshall.

1.8 Spanning Tree

Let $G = (V, E)$ be a graph. A spanning subgraph of a graph $G = (V, E)$ is a subgraph containing all vertices. A spanning subgraph that is a tree is called a spanning tree. A disconnected graph has no spanning tree. A connected graph may have many spanning trees.

A minimum spanning tree of a graph G is a spanning tree $T = (V, E_T)$ of the graph G whose sum of edge weights is minimal.

1.9 Kirchhoff's Matrix-Tree Theorem

Kirchhoff matrix. The matrix B of a graph G of size $n \times n$ is defined as follows:

$$B[i,j] = \begin{cases} -1, & \text{if vertices numbered } i \text{ and } j \text{ are adjacent;} \\ 0, & \text{if } i \neq j \text{ and the vertices are not adjacent;} \\ \deg(v_i), & \text{if } i = j. \end{cases}$$

Properties of the Kirchhoff matrix:

1. The sums of the elements in each row and each column of the matrix are equal to 0.
2. The cofactors of all elements of the matrix are equal to one another.
3. The determinant of the Kirchhoff matrix is zero.

Kirchhoff's matrix-tree theorem. The number of spanning trees in a connected graph G of order $n \geq 2$ is equal to the cofactor of any element of the Kirchhoff matrix B of the graph G.

1.10 Boruvka's Algorithm

Boruvka's algorithm is an algorithm for constructing a minimum spanning tree of a weighted connected undirected graph. It was proposed by Otakar Boruvka in 1926.

The algorithm works as follows:

1. Initially, each vertex forms a separate component, that is, a forest of isolated vertices.
2. For each component, the minimum-weight edge connecting it with another component is found.
3. All found edges are added to the spanning tree, and the corresponding components are merged.
4. Steps 2–3 are repeated until only one component remains.

At each iteration, the number of components decreases at least by half, so there are at most $O(\log V)$ iterations. The total complexity is $O(E \log V)$.

1.11 Prufer Coding

The Prufer encoding algorithm consists of the following steps. While the number of vertices is greater than two:

- Choose the leaf v with the smallest number.
- Add the number of the vertex adjacent to v to the Prufer code.
- Remove vertex v and its incident edge from the tree.

The obtained sequence is called the Prufer code of the given tree.

The decoding algorithm uses a Prufer code array of size $n-2$ and the array of all graph vertices $[1 \dots n]$. The following procedure is repeated $n-2$ times: the first element of the Prufer code is taken, and the smallest vertex not contained in the code is searched for in the vertex array. The found vertex and the current code element form an edge of the tree. These vertices are removed from the corresponding arrays. At the end, two vertices remain in the vertex array; they form the last edge of the tree. The complexity of the algorithms is $O(n)$.

1.12 Minimum Edge Cover

An **edge cover** of a graph $G = (V, E)$ is a subset of edges $S \subseteq E$ such that every vertex of the graph is incident to at least one edge from S. A **minimum edge cover** is an edge cover of minimum cardinality.

The relation with maximum matching is expressed by Gallai's theorem: for a graph without isolated vertices,

$$\alpha'(G) + \beta'(G) = |V(G)|,$$

where $\alpha'(G)$ is the size of a maximum matching and $\beta'(G)$ is the size of a minimum edge cover.

The algorithm for finding a minimum edge cover is:

1. Find a maximum matching M in the graph.
2. For each vertex not covered by matching M, add any edge incident to it to the cover.

The complexity is determined by the complexity of finding a maximum matching.

1.13 Eulerian Graph

An Eulerian graph is a graph that contains an Eulerian cycle, that is, a closed walk passing through all edges of the graph exactly once.

A graph is Eulerian if and only if it is connected and the degrees of all its vertices are even.

An Eulerian path is a path containing all edges. In this work, an Eulerian cycle is searched for. First, the program checks whether the graph is Eulerian. If it is not, the graph is modified so that the degree of every vertex becomes even.

To find an Eulerian cycle, an algorithm based on depth-first traversal is used:

- Choose an arbitrary vertex of the graph and put it on the stack.
- While the stack is not empty, repeat the following actions:
  - remove a vertex from the stack;
  - if this vertex has unvisited adjacent vertices, choose one of them and put it on the stack;
  - if the vertex has no unvisited adjacent vertices, add it to the Eulerian cycle list.
- Return to the Eulerian cycle list in reverse order.

The complexity of finding an Eulerian cycle is estimated as $O(V + E)$.

1.14 Fundamental Cutset System

A **cutset**, or cutting set, in a graph $G = (V, E)$ is a set of edges $C \subseteq E$ whose removal increases the number of connected components of the graph.

A **fundamental cutset** with respect to a spanning tree T is the cutset corresponding to one tree edge $t \in E_T$: when t is removed, the tree T splits into two components T_1 and T_2; the fundamental cutset C_t is the set of all edges of the original graph G whose one endpoint lies in T_1 and the other endpoint lies in T_2.

The **fundamental cutset system** is constructed for a selected spanning tree T and contains exactly $|V| - 1$ cutsets, one for each tree edge.

All other cutsets of the graph can be obtained using the symmetric difference operation on fundamental cutsets:

$$C = C_{t_1} \triangle C_{t_2} \triangle \cdots \triangle C_{t_k},$$

where $\triangle$ is the symmetric difference operation on sets.

2 Implementation Features

2.1 Project Structure

The program is implemented in C++17 and has a modular structure. Common data structures and generation functions are placed in the `shared` directory, the algorithms are divided by laboratory works, and the `lab_combined` directory contains the combined console program with a menu. The full source code is given in Appendices A–E.

```
graph-theory-labs/
|-- CMakeLists.txt
|-- README.md
|-- shared/
|   |-- constants.h
|   |-- graph.h
|   |-- graph.cpp
|   |-- distribution.h
|   |-- distribution.cpp
|   |-- dag_generator.h
|   |-- dag_generator.cpp
|   |-- weight_matrix.h
|   \-- weight_matrix.cpp
|-- lab1/
|   |-- eccentricity.h
|   |-- eccentricity.cpp
|   |-- shimbell.h
|   |-- shimbell.cpp
|   |-- path_counter.h
|   \-- path_counter.cpp
|-- lab2/
|   |-- dfs.h
|   |-- dfs.cpp
|   |-- floyd_warshall.h
|   \-- floyd_warshall.cpp
|-- lab3/
|   |-- ford_fulkerson.h
|   |-- ford_fulkerson.cpp
|   |-- min_cost_flow.h
|   \-- min_cost_flow.cpp
|-- lab4/
|   |-- kirchhoff.h
|   |-- kirchhoff.cpp
|   |-- boruvka.h
|   |-- boruvka.cpp
|   |-- edge_cover.h
|   |-- edge_cover.cpp
|   |-- prufer.h
|   \-- prufer.cpp
|-- lab5/
|   |-- eulerian.h
|   |-- eulerian.cpp
|   |-- fundamental_cutsets.h
|   \-- fundamental_cutsets.cpp
\-- lab_combined/
    |-- CMakeLists.txt
    \-- main.cpp
```

Listing 1. Project structure

The `shared` directory contains the graph class, the normal distribution generator, the graph

generator, and the weight matrix generator. Directories `lab1`–`lab5` contain the algorithms of the corresponding laboratory works. The file `lab_combined/main.cpp` connects all modules into a single user menu.

2.2 Class Graph

The `Graph` class is the basic structure for storing a directed or undirected graph. Its declaration is shown in Listing 3, and the method implementation is shown in Listing 4. The class stores the number of vertices `n`, the directedness flag `directed`, and the adjacency matrix `adj`.

The function `Graph(int n, bool directed)` creates an empty graph of the required type.

Input: the number of vertices `n` and the logical directedness flag `directed`.

Output: a graph object with an $n \times n$ adjacency matrix filled with zeros.

The function `addEdge` adds an edge or an arc between two vertices. If the graph is undirected, the adjacency matrix is filled symmetrically.

Input: vertex numbers `u` and `v`.

Output: the modified adjacency matrix of the graph.

The function `hasEdge` checks whether an edge or an arc exists.

Input: vertex numbers `u` and `v`.

Output: `true` if the edge exists; otherwise `false`.

The function `neighbors` returns all vertices adjacent to the specified vertex. For a directed graph it returns outgoing neighbors, and for an undirected graph it returns all neighboring vertices.

Input: vertex number `u`.

Output: a vector of neighboring vertex numbers.

The functions `printAdjMatrix` and `printFull` are used to print the graph to the console. The first one prints only the adjacency matrix, and the second one also prints the adjacency list, the edge list, and, if available, the weight matrix.

Input: a graph object and, for `printFull`, an optional weight matrix.

Output: a textual representation of the graph in the console.

2.3 Distribution, Graph and Weight Matrix Generation

Distribution generation functions are described in Listings 5 and 6. Generation uses an approximate normal distribution formula through the sum of uniformly distributed random variables.

The function `normalSample` generates one random value approximately distributed according to the normal law.

Input: the number of uniform random variables `n` used in the approximation.

Output: a real number corresponding to the approximate normal distribution.

The function `sampleDegree` converts a normal-distribution value into an admissible vertex degree.

Input: the maximum admissible degree `maxDeg`.

Output: an integer from 1 to `maxDeg`.

The function `generateDegreeSequence` builds a degree sequence for the specified number of vertices. The sum of degrees is adjusted to $2(n-1)$, which corresponds to a connected acyclic undirected graph.

Input: the number of vertices `numVertices`.

Output: a vector of vertex degrees.

Graph generation is implemented in `dag_generator.h` and `dag_generator.cpp`, shown in Listings 7 and 8. The main function `generateDAG` creates either a directed acyclic graph or

an undirected graph obtained from the directed model.

Input: the number of vertices `n` and the directedness flag `directed`.

Output: a `Graph` object with a generated adjacency matrix.

For a directed graph, a random topological order is built first. Then a path ensuring connectivity is added, after which additional arcs are added only forward according to the topological order. Therefore, the graph remains acyclic. For an undirected graph, arc directions are ignored and the graph becomes symmetric.

Weight matrix generation is implemented in Listings 9 and 10. The function `generateWeightMatrix` fills weights only for existing graph edges.

Input: graph `g` and weight generation mode `mode`.

Output: a weight matrix where absent edges are represented by `INF`.

Mode `POSITIVE` creates only positive weights, `NEGATIVE` creates only negative weights, and `MIXED` creates mixed weights. For an undirected graph, weights are written symmetrically.

2.4 Laboratory Work No. 1

The first laboratory work implements graph metric calculation, Shimbell's method, and route search. The code is located in `eccentricity.cpp`, `shimbell.cpp`, and `path_counter.cpp`; the full texts are given in Listings 13, 15, and 17.

The function `bfs` performs breadth-first search from a specified vertex and is used to calculate distances to all other vertices.

Input: graph `g`, start vertex `src`.

Output: a vector of distances from vertex `src`; value -1 means that a vertex is unreachable.

The function `computeMetrics` calculates eccentricities, radius, diameter, center, and diametral vertices of the graph. For each vertex it runs `bfs`, and then selects the maximum reachable distance from the obtained distances.

Input: graph `g`.

Output: the `GraphMetrics` structure containing the distance matrix, eccentricities, radius, diameter, center, and diametral vertices.

The function `shimbell` implements Shimbell's method for paths consisting of exactly k edges. For minimum paths, tropical multiplication with minimum selection is used; for maximum paths, maximum selection is used.

Input: weight matrix `W`, number of edges `k`.

Output: a pair of matrices: the minimum-path matrix and the maximum-path matrix.

The function `findAllPaths` searches for all simple routes between two vertices. The implementation uses recursive depth-first search with a visited-vertex array to prevent a vertex from being repeated inside one route.

Input: graph `g`, start vertex `src`, destination vertex `dst`.

Output: a vector of routes, where each route is represented by a sequence of vertices.

The functions `pathExists` and `countPaths` are auxiliary wrappers. The first checks whether at least one route exists, and the second returns the number of found routes. The function `printPaths` prints routes to the console.

Input: a graph or an already found list of routes.

Output: a logical value, the number of routes, or textual output in the console.

2.5 Laboratory Work No. 2

The second laboratory work implements depth-first graph traversal and the Floyd-Warshall algorithm. The code is given in Listings 19 and 21.

The function `dfs` performs depth-first traversal from a specified vertex. The implementation uses a stack, so the algorithm works iteratively. The number of iterations is also counted.

Input: graph `g`, start vertex `start`.

Output: traversal order, list of unreachable vertices, and the number of iterations printed to the console.

The function `floydWarshall` finds shortest paths between all pairs of vertices. Matrix `T` stores distances, and matrix `H` stores information for path reconstruction.

Input: graph `g`, weight matrix `W`.

Output: the `FloydResult` structure containing the distance matrix, the path reconstruction matrix, the number of iterations, and the negative-cycle flag.

The function `printFloydResult` prints the algorithm result and reconstructs the path between two vertices selected by the user. If a path does not exist, the program prints the corresponding message.

Input: the Floyd-Warshall result, the original weight matrix, the number of vertices, the start vertex, and the destination vertex.

Output: matrices `C`, `T`, `H`, the shortest path, and its length in the console.

2.6 Laboratory Work No. 3

The third laboratory work implements generation of capacity and cost matrices, the Ford-Fulkerson algorithm, and minimum-cost flow search. The code is located in Listings 23 and 25.

The function `generateCapacityMatrix` creates a capacity matrix for existing arcs of a directed graph.

Input: directed graph `g`.

Output: a capacity matrix where zero means that an arc is absent.

The function `generateCostMatrix` creates a matrix of the cost of passing one unit of flow through each arc.

Input: directed graph `g`.

Output: a cost matrix corresponding to the graph structure.

The function `fordFulkerson` implements the Ford-Fulkerson algorithm in the Edmonds-Karp variant, that is, an augmenting path is searched for using breadth-first search. At each step, the path capacity is found, and then the residual network is updated.

Input: number of vertices, capacity matrix, source `src`, sink `dst`.

Output: the `FFResult` structure containing the flow matrix and the maximum flow value.

The function `fordFulkersonSilent` performs the same algorithm but without printing intermediate paths. It is used inside the minimum-cost flow algorithm.

Input: number of vertices, capacity matrix, source and sink.

Output: the maximum flow value and the flow matrix without console output.

The function `minCostFlow` computes the minimum-cost flow for the value $\lfloor 2/3 \cdot \varphi_{\max} \rfloor$. First, the maximum flow is found, and then the residual network repeatedly selects shortest paths by cost found by the Floyd-Warshall algorithm.

Input: number of vertices, capacity matrix, cost matrix, source and sink.

Output: the `MCFResult` structure containing the maximum flow, required flow, achieved flow, total cost, and flow matrix.

The function `printMCFResult` prints the resulting minimum-cost flow and a verification calculation of the cost by arcs.

Input: the minimum-cost flow result, the capacity matrix, and the cost matrix.

Output: a table of arcs with positive flow and the total cost.

2.7 Laboratory Work No. 4

The fourth laboratory work implements Kirchhoff's matrix-tree theorem, Boruvka's algorithm, minimum edge cover, and Prufer code. The full code is given in Listings 27, 29, 31,

and 33.

The function `countSpanningTrees` counts the number of spanning trees of a graph. For this purpose, the Kirchhoff matrix is built, the first row and the first column are removed, and the determinant of the resulting matrix is calculated.

Input: undirected graph `g`.

Output: the number of spanning trees of the graph.

The function `boruvka` builds a minimum spanning tree. At each iteration, the minimum outgoing edge is selected for each component, and then the components are merged.

Input: undirected graph `g`, weight matrix `W`.

Output: the `BoruvkaResult` structure containing the edges of the minimum spanning tree and its total weight.

The function `minEdgeCover` finds a minimum edge cover. First, a maximum matching is constructed; then incident edges are added for uncovered vertices.

Input: graph `g`, weight matrix, flag selecting the spanning tree or original graph, and the list of spanning tree edges.

Output: the `EdgeCoverResult` structure containing cover edges and the maximum matching size.

The function `pruferEncode` encodes the minimum spanning tree by the Prufer code. At each step, the leaf with the smallest number is removed, and its neighbor is written to the code. The weight of the removed edge is stored separately.

Input: list of spanning tree edges, number of vertices.

Output: the Prufer code and the list of edge weights in removal order.

The function `pruferDecode` restores a tree from the Prufer code and the saved weights. At each step, the minimum leaf absent from the current code is selected.

Input: Prufer code, list of weights, number of vertices.

Output: list of edges of the restored tree with weights.

2.8 Laboratory Work No. 5

The fifth laboratory work implements checking a graph for Eulerian property, building an Eulerian cycle, and constructing the fundamental cutset system. The code is given in Listings 35 and 37.

The function `buildEulerianCycle` checks graph connectivity and parity of all vertex degrees. If the graph is not Eulerian, the program attempts to modify it by adding or removing edges so that the degrees become even and connectivity is preserved.

Input: undirected graph `g`.

Output: the `EulerianResult` structure containing initial degrees, odd vertices, the list of changes, and the constructed Eulerian cycle.

The function `hierholzer` builds an Eulerian cycle after the graph has become Eulerian. The algorithm sequentially removes traversed edges and forms the cycle in reverse order.

Input: adjacency matrix of an Eulerian graph.

Output: sequence of vertices of the Eulerian cycle.

The function `buildFundamentalCutsets` constructs a fundamental cutset system with respect to the minimum spanning tree. For each spanning tree edge, the edge is temporarily removed; then two tree components are found, and all original graph edges connecting these components are selected.

Input: original graph `g`, list of minimum spanning tree edges.

Output: a vector of fundamental cutsets.

The function `printCutsetSymmetricDifference` computes the symmetric difference of selected fundamental cutsets. An edge that appears twice is removed from the result, while an edge that appears once remains.

Input: list of fundamental cutsets and numbers of selected cutsets.

Output: the set of edges obtained by the symmetric difference operation.

2.9 Function main and Program Menu

All laboratory works are combined in the file `lab_combined/main.cpp`, whose full code is given in Listing 11. The program uses global variables to store the current graph, the weight matrix, capacity and cost matrices, the minimum spanning tree, and the Prufer encoding result.

The function `readInt` provides safe integer input. If the user enters invalid data, the input stream is cleared and the request is repeated.

Input: prompt string `prompt`.

Output: a correctly read integer.

The functions `requireGraph`, `requireWeight`, `requireFlowMatrices`, and `requireMST` check whether the required data has been prepared before an algorithm is launched.

Input: the current program state.

Output: an error message if the required data is missing.

Menu functions such as `menuGenerateGraph`, `menuShimbell`, `menuFloydWarshall`, `menuFordFulkerson`, `menuBoruvka`, and others read user parameters, check the correctness of the program state, and call the corresponding algorithms.

Input: the user's choice and additional values entered in the console.

Output: launch of the selected algorithm and printing of the result to the console.

The function `main` organizes the main program loop. Until the user selects the exit option, the menu is displayed, the command number is read, and the corresponding handler function is called.

Input: user commands from the console menu.

Output: sequential execution of laboratory algorithms until program termination.

Thus, the combined program makes it possible to perform the laboratory works in one interface: first the graph is generated, then the required matrices are built, and after that the user can launch algorithms from different sections of the work.

2.10 Detailed Description of Functions

Below is a more detailed description of the main functions of the program. For each function, its full name is given exactly as it is used in the source code, followed by its purpose, input data, and result. The source code is not duplicated in this section because it is fully provided in Appendices A–E; here the code is used as the basis for explaining the implementation. The function names in this subsection are hyperlinks to the corresponding source code listings.

Common Data Structures and Generation

Function `Graph::Graph()`

The default constructor creates an empty graph object. It is needed for global variables and for cases where the graph is generated later through the program menu. Inside the constructor, the number of vertices is set to zero, and the graph is considered undirected until a new object is explicitly created.

Input: no input parameters.

Output: an empty `Graph` object for which the method `isEmpty()` returns `true`.

Function `Graph::Graph(int n, bool directed)`

The main constructor of the `Graph` class. It creates an adjacency matrix of size $n \times n$ and fills it with zeros. The parameter `directed` determines whether the graph is directed. Since the graph is stored as an adjacency matrix, checking whether an edge exists takes constant time.

Input: the number of vertices `n`; logical parameter `directed`, equal to `true` for a directed graph and `false` for an undirected graph.

Output: a graph object with a prepared adjacency matrix and a stored graph type.

Function `Graph::addEdge(int u, int v)`

The method adds an edge between vertices `u` and `v`. If the graph is directed, only the element `adj[u][v]` is set in the adjacency matrix. If the graph is undirected, the symmetric element `adj[v][u]` is also set, so one edge is stored in two matrix cells.

Input: vertex numbers `u` and `v` in internal zero-based numbering.

Output: the modified adjacency matrix of the current `Graph` object.

Function `Graph::hasEdge(int u, int v) const`

The method checks whether an edge or arc from vertex `u` to vertex `v` exists. The implementation accesses one cell of the adjacency matrix and compares it with one.

Input: vertex numbers `u` and `v`.

Output: `true` if `adj[u][v] == 1`; otherwise `false`.

Function `Graph::neighbors(int u) const`

The method forms a list of all vertices reachable from vertex `u` in one step. For a directed graph, these are outgoing neighbors; for an undirected graph, these are all adjacent vertices because the adjacency matrix is symmetric.

Input: vertex number `u`.

Output: a `std::vector<int>` vector with neighboring vertex numbers.

Function `Graph::printAdjMatrix() const`

The method prints the adjacency matrix to the console. During output, vertices are converted from internal numbering $0, 1, \ldots, n-1$ to user numbering $1, 2, \ldots, n$. For an absent edge, `inf` is printed; for an existing edge, `1` is printed.

Input: the current `Graph` object.

Output: the adjacency matrix table in the console.

Function `Graph::printFull(const std::vector<std::vector<double>*> weightMatrix) const`

The method prints an extended representation of the graph: adjacency list, edge list, adjacency matrix, and, if passed, the weight matrix. The pointer to the weight matrix is optional, so the same function is used both before and after weight generation.

Input: the current graph and an optional pointer `weightMatrix` to the weight matrix.

Output: a complete textual description of the graph in the console.

Function `Graph::isEmpty() const`

The method is used to quickly check whether a graph has already been created. It returns true if the number of vertices is zero.

Input: the current `Graph` object.

Output: a logical value showing whether the graph is empty.

Function `double normalSample(int n)`

The function generates one value approximately distributed according to the normal law. It uses the sum of `n` uniformly distributed random variables and normalization according to the formula shown in the source file comments. This approach is convenient because it is simple and does not require a separate complex normal-distribution generator.

Input: number `n`, which sets the number of uniform random variables for approximation.

Output: a `double` value imitating a sample from the normal distribution.

Function `int sampleDegree(int maxDeg)`

The function converts a random normal-distribution value into an integer vertex degree. The obtained number is limited to the range from `1` to `maxDeg`, so every vertex receives an admissible positive degree.

Input: maximum admissible degree `maxDeg`.

Output: an integer representing a vertex degree.

Function `std::vector<int> generateDegreeSequence(int numVertices)`

The function creates a degree sequence for the specified number of vertices. First, a degree is generated for each vertex; then the degree sum is adjusted so that it corresponds to the required graph generation model. This sequence is later used when constructing a directed acyclic graph.

Input: number of vertices `numVertices`.

Output: a vector of integers where each element corresponds to the degree of one vertex.

Function `Graph generateDAG(int n, bool directed)`

The function is the main graph generation entry point. If `directed` is `true`, a directed acyclic graph is built. A random topological order is created, a path for connectivity is added, and additional arcs are placed only forward in this order. If `directed` is `false`, a directed model is built first, then edge directions are ignored, producing an undirected graph.

Input: number of vertices `n`; directedness flag `directed`.

Output: the generated `Graph` object.

Function `Matrix generateWeightMatrix(const Graph& g, WeightMode mode)`

The function creates a weight matrix for an already generated graph. A weight is assigned only where an edge exists in the graph. For absent edges, a special infinity value is used, which is important for Shimbell's method and the Floyd-Warshall algorithm. Mode `POSITIVE` creates positive weights, `NEGATIVE` creates negative weights, and `MIXED` creates mixed weights.

Input: graph `g`; weight generation mode `mode`.

Output: a `Matrix` of size $n \times n$ with weights of existing edges.

Functions of Laboratory Work No. 1

Function `std::vector<int> bfs(const Graph& g, int src)`

The function performs breadth-first search from vertex `src`. It is used as an auxiliary part of calculating graph metrics. The search follows the neighbor list obtained from the adjacency matrix. If a vertex is reachable, the shortest distance in the number of edges is written for it; if a vertex is unreachable, the value remains `-1`.

Input: graph `g`; start vertex `src`.

Output: a vector of distances from vertex `src` to all other vertices.

Function `GraphMetrics computeMetrics(const Graph& g)`

The function calculates the main metric characteristics of a graph. For each vertex, `bfs` is called, after which a distance matrix is formed. The eccentricity of a vertex is the maximum distance from it to reachable vertices. Then the radius, diameter, center vertices, and diametral vertices are selected from the eccentricities.

Input: graph `g`.

Output: the `GraphMetrics` structure containing the distance matrix, eccentricities, radius, diameter, center, and diametral vertices.

Function `void printMetrics(const GraphMetrics& m)`

The function prints the results obtained in `computeMetrics`: the distance matrix, eccentricities, radius, diameter, center, and diametral vertices.

Input: the `GraphMetrics` structure.

Output: formatted output of metric characteristics in the console.

Function `std::pair<Matrix, Matrix> shimbell(const Matrix& W, int k)`

The function implements Shimbell's method for finding paths consisting of exactly `k` edges. Internally, tropical matrix multiplication is used: for minimum paths, the minimum sum of weights is selected; for maximum paths, the maximum is selected. For `k = 0`, an analogue of the identity matrix is returned, where the path from a vertex to itself has length zero.

Input: weight matrix `W`; number of edges in the path `k`.

Output: a pair of matrices: the first contains minimum path weights, and the second contains maximum path weights for exactly `k` steps.

Function `void printMatrix(const Matrix& M)`

The function prints a weight matrix to the screen. Large positive values are displayed as `INF`, and large negative values are displayed as `-INF`.

Input: matrix `M`.

Output: a table of matrix values in the console.

Function `std::vector<std::vector<int» findAllPaths(const Graph& g, int src, int dst)`

The function finds all simple routes between vertices `src` and `dst`. Recursive depth-first search is used. The visited-vertex array prevents the same vertex from being repeated in one route.

Input: graph `g`; start vertex `src`; destination vertex `dst`.

Output: a list of routes, where each route is represented by a vector of vertex numbers.

Function `bool pathExists(const Graph& g, int src, int dst)`

The function checks whether at least one path exists between the selected vertices.

Input: graph `g`; start vertex `src`; destination vertex `dst`.

Output: `true` if a path exists; otherwise `false`.

Function `int countPaths(const Graph& g, int src, int dst)`

The function counts the number of simple routes between two vertices.

Input: graph `g`; start vertex `src`; destination vertex `dst`.

Output: an integer equal to the number of found routes.

Function `void printPaths(const std::vector<std::vector<int»& paths, int src, int dst)`

The function prints the found routes using user vertex numbering. If the list is empty, it reports that there are no routes between the selected vertices.

Input: list of routes `paths`; start vertex `src`; destination vertex `dst`.

Output: a textual list of routes in the console.

Functions of Laboratory Work No. 2

Function `void dfs(const Graph& g, int start)`

The function performs depth-first traversal of the graph starting from vertex `start`. The implementation uses a stack, records the visiting order, counts iterations, and prints unreachable vertices.

Input: graph `g`; start vertex `start`.

Output: traversal order, list of unreachable vertices, and number of iterations in the console.

Function `FloydResult floydWarshall(const Graph& g, const std::vector<std::vector<double» W)`

The function implements the Floyd-Warshall algorithm for finding shortest paths between all pairs of vertices. Matrix `T` stores current distances, and matrix `H` is used to reconstruct paths. The diagonal elements are checked to detect negative cycles.

Input: graph `g`; weight matrix `W`, where the diagonal is zero for this algorithm.

Output: the `FloydResult` structure with the distance matrix, path reconstruction matrix, number of iterations, and negative-cycle flag.

Function `void printFloydResult(const FloydResult& res, const std::vector<std::vector<double»& W, int n, int src, int dst)`

The function prints the original weight matrix, the final shortest-distance matrix, the path reconstruction matrix, and the path between vertices selected by the user.

Input: result `res`; weight matrix `W`; number of vertices `n`; start vertex `src`; destination vertex `dst`.

Output: detailed output of tables and the reconstructed shortest path in the console.

Functions of Laboratory Work No. 3

Function `std::vector<std::vector<int» generateCapacityMatrix(const Graph& g)`
The function creates a capacity matrix for a directed graph. For each existing arc, an integer capacity is generated, and absent arcs remain zero.

Input: directed graph `g`.

Output: a capacity matrix where a positive number means that an arc exists and gives its capacity.

Function `std::vector<std::vector<int» generateCostMatrix(const Graph& g)`
The function creates a cost matrix for the same arcs that are present in the graph.

Input: directed graph `g`.

Output: a cost matrix where zero for an absent arc means that flow along it is not allowed.

Function `FFResult fordFulkerson(int n, const std::vector<std::vector<int»& cap, int src, int dst)`
The function implements the Ford-Fulkerson algorithm in the Edmonds-Karp variant. Every augmenting path is found by BFS, then the bottleneck capacity is used to increase the flow and update reverse arcs.

Input: number of vertices `n`; capacity matrix `cap`; source `src`; sink `dst`.

Output: the `FFResult` structure containing the final flow matrix, maximum flow value, source, and sink.

Function `FFResult fordFulkersonSilent(int n, const std::vector<std::vector<int»& cap, int src, int dst)`
The function performs the same maximum-flow calculation as `fordFulkerson`, but it does not print intermediate augmenting paths.

Input: number of vertices `n`; capacity matrix `cap`; source `src`; sink `dst`.

Output: the `FFResult` structure without intermediate console output.

Function `MCFResult minCostFlow(int n, const std::vector<std::vector<int»& cap, const std::vector<std::vector<int»& cost, int src, int dst)`
The function computes the minimum-cost flow. First, the maximum flow is found using `fordFulkersonSilent`; then the target flow is set to $\lfloor 2/3 \cdot \varphi_{max} \rfloor$. The flow is accumulated by adding shortest paths by cost in the residual network.

Input: number of vertices `n`; capacity matrix `cap`; cost matrix `cost`; source `src`; sink `dst`.

Output: the `MCFResult` structure with maximum flow, target flow, achieved flow, total cost, and flow matrix.

Functions of Laboratory Work No. 4

Function `long long countSpanningTrees(const Graph& g)`
The function counts the number of spanning trees using Kirchhoff's matrix-tree theorem by constructing the Laplacian matrix, deleting one row and one column, and computing the determinant.

Input: undirected graph `g`.

Output: the number of spanning trees as `long long`.

Function `BoruvkaResult boruvka(const Graph& g, const std::vector<std::vector<double»& W)`
The function implements Boruvka's algorithm. At each iteration, the cheapest outgoing edge is selected for each component; selected edges are added to the spanning tree and components are merged.

Input: undirected graph `g`; weight matrix `W`.

Output: the `BoruvkaResult` structure containing minimum spanning tree edges and total weight.

Function `EdgeCoverResult minEdgeCover(const Graph& g, const std::vector<std::vector<doub`

`W, bool useMST, const std::vector<MSTEdge>& mstEdges)`
The function builds a minimum edge cover either on the minimum spanning tree or on the full graph. It first finds a maximum matching and then adds incident edges for uncovered vertices.

Input: graph `g`; weight matrix `W`; flag `useMST`; list of spanning tree edges `mstEdges`.

Output: the `EdgeCoverResult` structure containing cover edges and matching size.

Function `PruferResult pruferEncode(const std::vector<MSTEdge>& mstEdges, int n)`
The function encodes a spanning tree using the Prufer code. It repeatedly removes the smallest leaf, records its neighbor, and saves the removed edge weight.

Input: list of spanning tree edges `mstEdges`; number of vertices `n`.

Output: the `PruferResult` structure containing the Prufer code and saved weights.

Function `std::vector<MSTEdge> pruferDecode(const std::vector<int>& code, const std::vector<double>& weights, int n)`
The function restores a tree from a Prufer code and the saved weights by repeatedly connecting the smallest leaf absent from the current code with the first code element.

Input: Prufer code `code`; list of weights `weights`; number of vertices `n`.

Output: list of edges of the restored tree.

Functions of Laboratory Work No. 5

Function `EulerianResult buildEulerianCycle(const Graph& g)`
The function checks whether an undirected graph is Eulerian and builds an Eulerian cycle. If the conditions are not satisfied, it tries to modify a local copy of the graph without changing the original object.

Input: undirected graph `g`.

Output: the `EulerianResult` structure containing original degrees, odd vertices, performed changes, and the found Eulerian cycle.

Function `std::vector<FundamentalCutset> buildFundamentalCutsets(const Graph& g, const std::vector<MSTEdge>& mstEdges)`
The function builds the fundamental cutset system with respect to the minimum spanning tree. For every tree edge, the edge is removed, two components are found, and all original graph edges connecting these components are collected.

Input: original graph `g`; list of minimum spanning tree edges `mstEdges`.

Output: a vector of `FundamentalCutset` structures.

Function `void printCutsetSymmetricDifference(const std::vector<FundamentalCutset>& cutsets, const std::vector<int>& selectedIndices)`
The function computes the symmetric difference of selected fundamental cutsets, keeping edges that occur an odd number of times.

Input: list of cutsets `cutsets`; numbers of selected cutsets `selectedIndices`.

Output: the set of edges obtained as the symmetric difference, printed to the console.

Functions of the Combined Menu

Function `static inline int D(int v)`
The function converts internal zero-based vertex numbering to user numbering starting from one.

Input: vertex number `v` in internal numbering.

Output: vertex number `v + 1` for console output.

Function `static int readInt(const std::string& prompt)`
The function safely reads an integer, repeats the request on invalid input, and clears the input stream.

Input: prompt string `prompt`.

Output: correctly read integer.

Functions `requireGraph()`, `requireWeight()`, `requireFlowMatrices()`, `requireMST()`
These functions print error messages if the user tries to launch an algorithm without the required preliminary data.

Input: current state of global program variables.

Output: diagnostic message in the console.

Functions `void menuGenerateGraph()`, `void menuGenerateWeightMatrix()`, `menuDFS()`, `menuFloydWarshall()`, `menuFordFulkerson()`, `menuBoruvka()`, `menuEulerianCycle()`
These menu functions read user parameters, check that the current program state is valid, and call the corresponding algorithmic functions.

Input: current graph, matrices, and parameters entered by the user.

Output: execution of the selected algorithm and result output in the console.

Function `int main()`
The main function organizes the general application loop. Until the user selects item `0`, the program prints the menu, reads the command number through `readInt`, and calls the corresponding menu function using the `switch` operator.

Input: user commands from the console.

Output: execution of the selected algorithms and program termination after the exit item is selected.

3 Program Results

3.1 Tasks 1–11: Directed Graph with 5 Vertices

After the program starts, the main menu is displayed, containing items for all five laboratory works. For tasks 1–11, a directed graph with 5 vertices was generated, because the first three laboratory works use a directed acyclic graph and algorithms for routes, shortest paths, and flows. The main program menu is shown in Fig. 1.

```
C:\Users\shami\OneDrive\Documents\theoryGraph\graph-theory-labs1\cmake-build-debug\lab_combined\lab_combined.exe

========================================
   GRAPH THEORY LABS
========================================
  1. Generate graph
  2. Show graph (adjacency list / matrix)
  3. Generate weight matrix
  =====Lab 1=====
  4. Eccentricities, center, diametral vertices
  5. Shimbell's method (min/max paths, k steps)
  6. Find all routes between two vertices
  =====Lab 2=====
  7. DFS traversal
  8. Shortest path: Floyd-Warshall
  =====Lab 3=====
  9.  Generate capacity and cost matrices
  10. Max flow: Ford-Fulkerson
  11. Min-cost flow
  =====Lab 4=====
  12. Kirchhoff: count spanning trees
  13. Boruvka: minimum spanning tree
  14. Minimum edge cover
  15. Prufer encode / decode
  =====Lab 5=====
  16. Eulerian check / modify / Euler cycle
  17. Fundamental cutsets from MST
  0.  Exit
----------------------------------------
  Enter choice:
```

Fig. 1. Main menu of the combined program

When item 1 is selected, the user enters the number of vertices and the graph type. For the first part of the results, a directed graph was selected. The program builds the adjacency matrix and reports that a weight matrix must be generated for further algorithms; see Fig. 2.

```
Enter choice:1


Number of vertices:5

Graph type:
  1. Directed
  2. Undirected
Choose [1/2]:1


------------------------------------------------------------
ADJACENCY MATRIX  [DIRECTED]
------------------------------------------------------------

         1    2    3    4    5
------------------------------
  1 |    0  inf  inf  inf  inf
  2 |    1    0    1    1    1
  3 |    1  inf    0    1    1
  4 |    1  inf  inf    0    1
  5 |    1  inf  inf  inf    0

------------------------------------------------------------
Graph with 5 vertices generated.
Generate weight matrix with option 3.
```

Fig. 2. Generation of a directed graph with 5 vertices

When item 2 is selected, the full graph representation is printed: adjacency list, arc list, and adjacency matrix. This makes it possible to check the structure of the generated directed graph before launching algorithms; see Fig. 3.

```
------------------------------------------------------------
ADJACENCY LIST  [DIRECTED]
------------------------------------------------------------

Vertex   1  -->  (isolated)   (degree=0)
Vertex   2  -->  1, 3, 4, 5   (degree=4)
Vertex   3  -->  1, 4, 5   (degree=3)
Vertex   4  -->  1, 5   (degree=2)
Vertex   5  -->  1   (degree=1)

EDGE LIST:
    2  ---->    1
    2  ---->    3
    2  ---->    4
    2  ---->    5
    3  ---->    1
    3  ---->    4
    3  ---->    5
    4  ---->    1
    4  ---->    5
    5  ---->    1

------------------------------------------------------------
ADJACENCY MATRIX  [DIRECTED]
------------------------------------------------------------

         1    2    3    4    5
------------------------------
  1 |    0  inf  inf  inf  inf
  2 |    1    0    1    1    1
  3 |    1  inf    0    1    1
  4 |    1  inf  inf    0    1
  5 |    1  inf  inf  inf    0

------------------------------------------------------------
```

Fig. 3. Output of the adjacency list, arc list, and adjacency matrix

When item `3` is selected, the user chooses the weight generation mode: positive, negative, or mixed. After that, the program prints the weight matrix, where INF means that an arc is absent; see Fig. 4.

```
Enter choice:3


Weight mode:
  1. Positive only
  2. Negative only
  3. Mixed
Choose [1/2/3]:3

Weight matrix generated.

            1       2       3       4       5
-----------------------------------------------
  1 |     INF     INF     INF     INF     INF
  2 |       1     INF      -1       4      -8
  3 |       1     INF     INF       6      -1
  4 |       3     INF     INF     INF       4
  5 |       2     INF     INF     INF     INF
```

Fig. 4. Weight matrix generation for the directed graph

When item 4 is selected, distances between vertices, eccentricities, radius, diameter, graph center, and diametral vertices are calculated. The algorithm result is represented as a distance matrix and a final table; see Fig. 5.

```
Enter choice:4



         1    2    3    4    5
-----------------------------
  1 |    0  inf  inf  inf  inf
  2 |    1    0    1    1    1
  3 |    1  inf    0    1    1
  4 |    1  inf  inf    0    1
  5 |    1  inf  inf  inf    0

Vertex  | Eccentricity | Role
--------+--------------+--------------------------
     1  |         0    | -
     2  |         1    | DIAMETRICAL + CENTER
     3  |         1    | DIAMETRICAL + CENTER
     4  |         1    | DIAMETRICAL + CENTER
     5  |         1    | DIAMETRICAL + CENTER

Radius   = 1
Diameter = 1

CENTER vertices      = { 2 3 4 5 }
DIAMETRICAL vertices = { 2 3 4 5 }
```

Fig. 5. Graph eccentricities, center, and diametral vertices

When item `5` is selected, the user specifies the number of steps k, after which the program applies Shimbell's method. The result contains matrices of minimum and maximum paths consisting of exactly the specified number of arcs; see Fig. 6.

```
Enter k (number of steps):2


------------------------------------------------------------
SHIMBELL MIN-path (k=2)
------------------------------------------------------------

           1       2       3       4       5
-------------------------------------------
  1 |     INF     INF     INF     INF     INF
  2 |      -6     INF     INF       5      -2
  3 |       1     INF     INF     INF      10
  4 |       6     INF     INF     INF     INF
  5 |     INF     INF     INF     INF     INF

------------------------------------------------------------
SHIMBELL MAX-path (k=2)
------------------------------------------------------------

           1       2       3       4       5
-------------------------------------------
  1 |    -INF    -INF    -INF    -INF    -INF
  2 |       7    -INF    -INF       5       8
  3 |       9    -INF    -INF    -INF      10
  4 |       6    -INF    -INF    -INF    -INF
  5 |    -INF    -INF    -INF    -INF    -INF
```

Fig. 6. Shimbell's method for the directed weighted graph

When item `6` is selected, the user enters the start and destination vertices. The program determines whether a route exists between them, counts the number of simple routes, and prints the found routes; see Fig. 7.

```
ADJACENCY MATRIX  [DIRECTED]
------------------------------------------------------------

        1    2    3    4    5
-----------------------------
  1 |    0  inf  inf  inf  inf
  2 |    1    0    1    1    1
  3 |    1  inf    0    1    1
  4 |    1  inf  inf    0    1
  5 |    1  inf  inf  inf    0

------------------------------------------------------------
WEIGHT MATRIX  (INF = no edge, 0 = self)
------------------------------------------------------------

           1       2       3       4       5
-------------------------------------------
  1 |     INF     INF     INF     INF     INF
  2 |       1     INF      -1       4      -8
  3 |       1     INF     INF       6      -1
  4 |       3     INF     INF     INF       4
  5 |       2     INF     INF     INF     INF

------------------------------------------------------------
Vertices are numbered 1 to 5.
Source vertex:2

Destination vertex:4


Path from 2 to 4: EXISTS  (2 routes)

Routes:
  Route 1:  { 2 -> 3 -> 4 }
  Route 2:  { 2 -> 4 }
```

Fig. 7. Search for all routes between two vertices

When item 7 is selected, depth-first graph traversal is performed from the specified vertex. The result contains the traversal order, the number of visited vertices, and the number of algorithm iterations; see Fig. 8.

```
Enter choice:7


Enter start vertex:2


------------------------------------------------------------
DFS TRAVERSAL  (start = vertex 2)
------------------------------------------------------------

Traversal order:
2 -> 1 -> 3 -> 4 -> 5

Vertices visited   : 5 / 5

Total iterations   : 23
```

Fig. 8. Depth-first graph traversal

When item `8` is selected, the user enters the start and destination vertices for shortest-path search. The Floyd-Warshall algorithm prints the original weight matrix, shortest-distance matrix, path reconstruction matrix, the path itself, and the number of iterations; see Fig. 9.

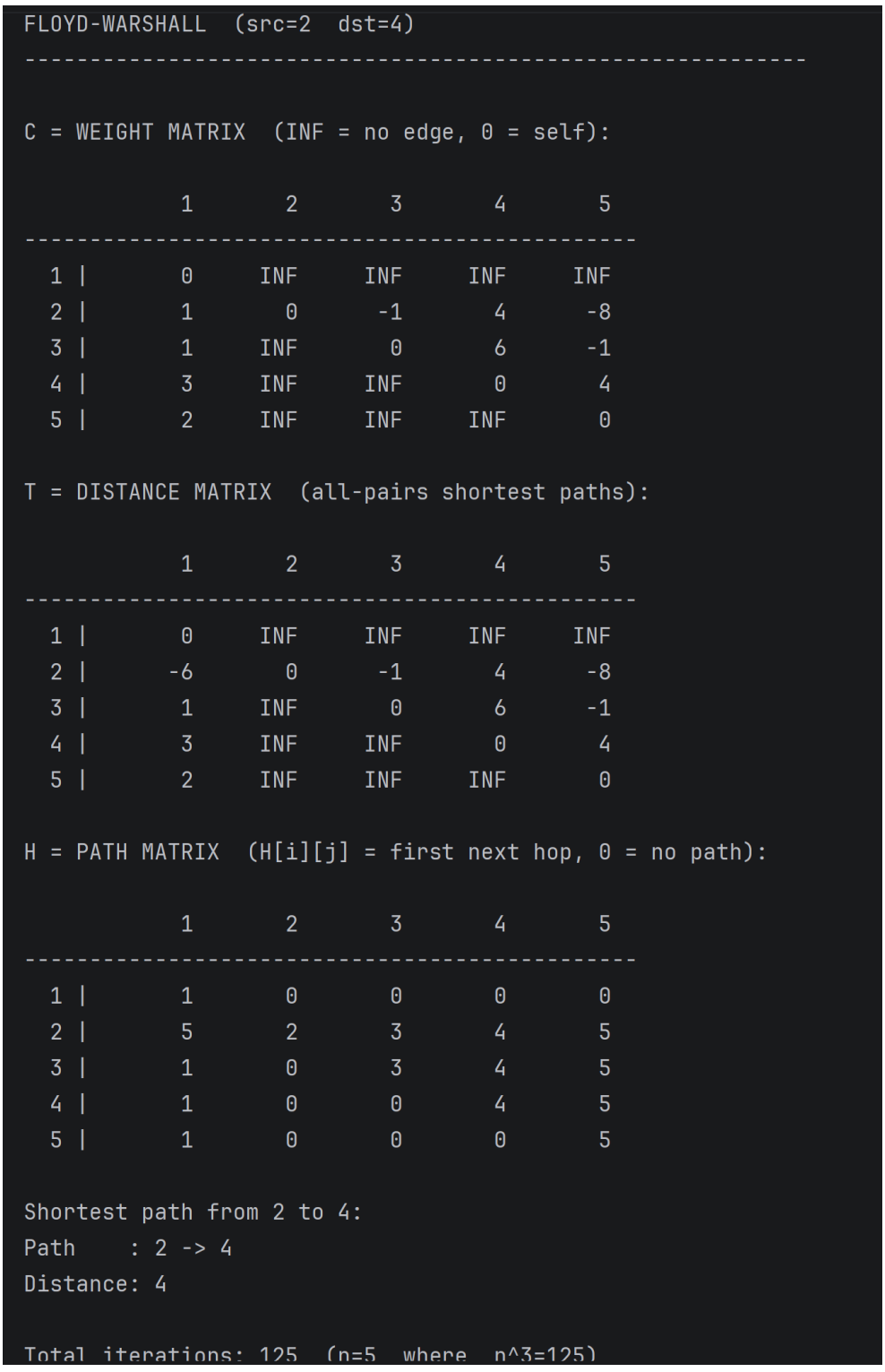

Fig. 9. Shortest path search using the Floyd-Warshall algorithm

When item 9 is selected, capacity and cost matrices are generated based on the directed graph. In addition, the program prints incoming and outgoing vertex degrees so that the user can choose the source and sink more easily; see Fig. 10.

```
Enter choice:9



CAPACITY MATRIX  Omega[i][j]:

          1     2     3     4     5
-------------------------------------
  1 |     0     0     0     0     0
  2 |     3     0     2    10     9
  3 |    20     0     0    11     6
  4 |    15     0     0     0    16
  5 |    14     0     0     0     0

COST MATRIX  D[i][j]:

          1     2     3     4     5
-------------------------------------
  1 |     0     0     0     0     0
  2 |     9     0     4    10     6
  3 |     1     0     0     1     3
  4 |     6     0     0     0     9
  5 |    10     0     0     0     0

Vertex degrees (to help choose source / sink):
  Vertex | Out-degree | In-degree
  ---------------------------------
      1   |      0     |     4
      2   |      4     |     0
      3   |      3     |     1
      4   |      2     |     2
      5   |      1     |     3
Good source: high out-degree, low in-degree.
Good sink:   high in-degree,  low out-degree.
```

Fig. 10. Generation of capacity and cost matrices

When item `10` is selected, the program finds the maximum flow using the Ford-Fulkerson algorithm. The screen displays augmenting paths, the maximum-flow value, and flow distribution by arcs; see Fig. 11.

```
Enter choice:10


Source vertex:2

Sink vertex:4

Path 1: 2 -> 4  (delta = 10)
Path 2: 2 -> 3 -> 4  (delta = 2)

------------------------------------------------------------
FORD-FULKERSON RESULT
Source: 2   Sink: 4
Maximum flow  phi_max = 12
------------------------------------------------------------

CAPACITY MATRIX  Omega[i][j]:

          1     2     3     4     5
-------------------------------------
  1 |     0     0     0     0     0
  2 |     3     0     2    10     9
  3 |    20     0     0    11     6
  4 |    15     0     0     0    16
  5 |    14     0     0     0     0

FLOW on each arc (only arcs with flow > 0):
  (2 -> 3)  flow=2  cap=2
  (2 -> 4)  flow=10  cap=10
  (3 -> 4)  flow=2  cap=11
```

Fig. 11. Finding the maximum flow using the Ford-Fulkerson algorithm

When item 11 is selected, the minimum-cost flow is computed. The required flow value is $\lfloor 2/3 \cdot \varphi_{\max} \rfloor$. The program prints the shortest paths by cost, the achieved flow, and the total cost; see Fig. 12.

```
Enter choice:11


Source vertex:2

Sink vertex:4

phi_max = 12   theta = floor(2/3 * 12) = 8
Step 1 [Floyd-Warshall]: 2 -> 3 -> 4  delta=2  path_cost=5  total_flow=2
Step 2 [Floyd-Warshall]: 2 -> 4  delta=6  path_cost=10  total_flow=8

------------------------------------------------------------
MIN-COST FLOW RESULT
phi_max        = 12
theta (2/3*max)= 8
Flow achieved  = 8
Total cost     = 70
------------------------------------------------------------

FLOW DETAILS (arcs with flow > 0):
  Arc       flow   cap   cost/unit   cost
  ------------------------------------------------------
  (2->3)       2     2           4       8
  (2->4)       6    10          10      60
  (3->4)       2    11           1       2
  ------------------------------------------------------
  Total cost = 70
```

Fig. 12. Minimum-cost flow calculation

3.2 Tasks 12–17: Undirected Graph with 7 Vertices

For tasks 12–17, an undirected graph with 7 vertices was generated, because the algorithms of the fourth and fifth laboratory works operate on an undirected graph. After generation, the adjacency matrix is printed; see Fig. 13.

```
Enter choice:1


Number of vertices:7

Graph type:
  1. Directed
  2. Undirected
Choose [1/2]:2


------------------------------------------------------------
ADJACENCY MATRIX  [UNDIRECTED]
------------------------------------------------------------

         1    2    3    4    5    6    7
---------------------------------------
  1 |    0    1  inf  inf    1    1    1
  2 |    1    0    1  inf    1    1    1
  3 |  inf    1    0    1    1  inf  inf
  4 |  inf  inf    1    0  inf    1    1
  5 |    1    1    1  inf    0    1    1
  6 |    1    1  inf    1    1    0    1
  7 |    1    1  inf    1    1    1    0

Note: Undirected => matrix is symmetric (mat[i][j] == mat[j][i])

------------------------------------------------------------
Graph with 7 vertices generated (undirected).
Generate weight matrix with option 3.
```

Fig. 13. Generation of an undirected graph with 7 vertices

When item 12 is selected, the program builds the Kirchhoff matrix, removes the first row and first column to obtain a cofactor, and calculates the determinant. The obtained value is the number of spanning trees of the graph; see Fig. 14.

```
------------------------------------------------------------
KIRCHHOFF'S MATRIX-TREE THEOREM
------------------------------------------------------------

Kirchhoff (Laplacian) matrix B:
B[i][i] = degree(i),  B[i][j] = -1 if edge {i,j}, else 0

         1    2    3    4    5    6    7
-----------------------------------------
  1 |    4   -1    0    0   -1   -1   -1
  2 |   -1    5   -1    0   -1   -1   -1
  3 |    0   -1    3   -1   -1    0    0
  4 |    0    0   -1    3    0   -1   -1
  5 |   -1   -1   -1    0    5   -1   -1
  6 |   -1   -1    0   -1   -1    5   -1
  7 |   -1   -1    0   -1   -1   -1    5

Cofactor A_11 — delete row 1 and col 1:

         2    3    4    5    6    7
-------------------------------------
  2 |    5   -1    0   -1   -1   -1
  3 |   -1    3   -1   -1    0    0
  4 |    0   -1    3    0   -1   -1
  5 |   -1   -1    0    5   -1   -1
  6 |   -1    0   -1   -1    5   -1
  7 |   -1    0   -1   -1   -1    5

det(A_11) = 1584
Number of spanning trees = 1584
```

Fig. 14. Counting spanning trees using Kirchhoff's theorem

When item 13 is selected, a minimum spanning tree is built using Boruvka's algorithm. The program prints connected components, added safe edges, and the final weight of the obtained spanning tree step by step; see Fig. 15.

```
Enter choice:13


--- Round 1 ---
Components: {1} {2} {3} {4} {5} {6} {7}
  Add edge (1, 2)  weight=1
  Add edge (2, 3)  weight=1
  Add edge (3, 4)  weight=1
  Add edge (1, 5)  weight=1
  Add edge (2, 6)  weight=1
  Add edge (1, 7)  weight=1

------------------------------------------------------------
BORUVKA MST RESULT
------------------------------------------------------------

MST edges:
  Arc        weight
  ----------------------
  (1 -> 2)    1
  (2 -> 3)    1
  (3 -> 4)    1
  (1 -> 5)    1
  (2 -> 6)    1
  (1 -> 7)    1

Total MST weight = 6
```

Fig. 15. Building a minimum spanning tree using Boruvka's algorithm

When item `14` is selected, the user chooses whether to search for a minimum edge cover on the original graph or on the constructed spanning tree. The result contains the size of the maximum matching and the edges of the minimum cover; see Fig. 16.

```
Enter choice:14


Work on:
  1. MST (requires Boruvka, option 13)
  2. Full graph
Choose [1/2]:1


------------------------------------------------------------
MINIMUM EDGE COVER
------------------------------------------------------------

Maximum matching  |M| = 3
Min edge cover   |C| = 4  (= n - |M| when no isolated vertices)

Cover edges:
  Arc        weight
  ----------------------
  (1 -- 5)    1
  (2 -- 6)    1
  (3 -- 4)    1
  (7 -- 1)    1
```

Fig. 16. Finding a minimum edge cover

When item 15 is selected, the constructed minimum spanning tree is encoded by the Prufer code with weight preservation, and then reverse decoding is performed. The screen displays removed leaves, the Prufer code, edge weights, and the restored tree; see Fig. 17.

```
  Remove leaf 4  neighbor=3  weight=1
  Remove leaf 3  neighbor=2  weight=1
  Remove leaf 5  neighbor=1  weight=1
  Remove leaf 6  neighbor=2  weight=1
  Remove leaf 2  neighbor=1  weight=1
  Last edge: (1, 7)  weight=1

 ------------------------------------------------------------
 PRUFER CODE  (n=7, code length=5)
 ------------------------------------------------------------

Prufer sequence: [ 3, 2, 1, 2, 1 ]

 Edge weights (in removal order, length 6):
 [ 1, 1, 1, 1, 1, 1 ]
 (last weight is for the final edge not in Prufer sequence)

 ------------------------------------------------------------
  Connect 4 -- 3  weight=1
  Connect 3 -- 2  weight=1
  Connect 5 -- 1  weight=1
  Connect 6 -- 2  weight=1
  Connect 2 -- 1  weight=1
  Connect 1 -- 7  weight=1  (final edge)

 ------------------------------------------------------------
 DECODED TREE EDGES
 ------------------------------------------------------------
  (4, 3)  weight=1
  (3, 2)  weight=1
  (5, 1)  weight=1
  (6, 2)  weight=1
  (2, 1)  weight=1
  (1, 7)  weight=1

 Total weight = 6
```

Fig. 17. Encoding and decoding the spanning tree using the Prufer code

When item `16` is selected, the program checks whether the graph is Eulerian. If the graph does not satisfy the even-degree condition, edge modification is performed; after that, an Eulerian cycle is built; see Fig. 18.

```
LAB 5: EULERIAN GRAPH CHECK AND EULER CYCLE
------------------------------------------------------------

Connected: YES
Original graph is Eulerian: NO

Original degrees:
  Vertex | Degree | Parity
  -------------------------
      1   |    4   | even
      2   |    5   | odd
      3   |    3   | odd
      4   |    3   | odd
      5   |    5   | odd
      6   |    5   | odd
      7   |    5   | odd

Modification log:
  - Odd vertices: 2 3 4 5 6 7
  - Added missing edge (2 -- 4).
  - Odd vertices: 3 5 6 7
  - Added missing edge (3 -- 6).
  - Odd vertices: 5 7
  - Removed edge (5 -- 7); graph remains connected.

Modified degrees:
  deg(1) = 4  even
  deg(2) = 6  even
  deg(3) = 4  even
  deg(4) = 4  even
  deg(5) = 4  even
  deg(6) = 6  even
  deg(7) = 4  even

Euler cycle:
  1 -> 2 -> 3 -> 4 -> 2 -> 5 -> 1 -> 6 -> 2 -> 7 -> 4 -> 6 -> 3 -> 5 -> 6 -> 7 -> 1
```

Fig. 18. Checking Eulerian property, modification, and Eulerian cycle construction

When item 17 is selected, the program builds a minimum spanning tree using Boruvka's algorithm and then obtains a fundamental cutset for every edge of the spanning tree. After printing the fundamental system, the user selects cutset numbers, and the program computes their symmetric difference; see Fig. 19 and Fig. 20.

```
Enter choice:17


Building MST with Boruvka (Lab 4 algorithm)...
--- Round 1 ---
Components: {1} {2} {3} {4} {5} {6} {7}
  Add edge (1, 2)  weight=1
  Add edge (2, 3)  weight=1
  Add edge (3, 4)  weight=1
  Add edge (1, 5)  weight=1
  Add edge (2, 6)  weight=1
  Add edge (1, 7)  weight=1

------------------------------------------------------------
BORUVKA MST RESULT
------------------------------------------------------------

MST edges:
  Arc        weight
  ----------------------
  (1 -> 2)    1
  (2 -> 3)    1
  (3 -> 4)    1
  (1 -> 5)    1
  (2 -> 6)    1
  (1 -> 7)    1

Total MST weight = 6
```

Fig. 19. Construction of the fundamental cutset system

```
------------------------------------------------------------
LAB 5: FUNDAMENTAL CUTSET SYSTEM (EVEN VARIANT)
------------------------------------------------------------

Cutset 1  for tree edge (1 -- 2):
  { (1--2), (1--6), (2--5), (2--7), (3--5), (4--7), (5--6), (6--7) }
Cutset 2  for tree edge (2 -- 3):
  { (2--3), (3--5), (4--6), (4--7) }
Cutset 3  for tree edge (3 -- 4):
  { (3--4), (4--6), (4--7) }
Cutset 4  for tree edge (1 -- 5):
  { (1--5), (2--5), (3--5), (5--6), (5--7) }
Cutset 5  for tree edge (2 -- 6):
  { (1--6), (2--6), (4--6), (5--6), (6--7) }
Cutset 6  for tree edge (1 -- 7):
  { (1--7), (2--7), (4--7), (5--7), (6--7) }

------------------------------------------------------------
Number of cutsets for symmetric difference:2

Cutset number:1

Cutset number:1


Symmetric difference of selected cutsets (1, 1):
  {  }
```

Fig. 20. Symmetric difference of selected fundamental cutsets

3.3 Error Handling

The program checks the correctness of user actions. If the user tries to call an algorithm before generating a graph or a weight matrix, the program prints a message indicating the required menu item; see Fig. 21.

```
Enter choice:12


Error: Lab 4 requires an undirected graph.
Please generate an undirected graph (option 1, choose 2).
```

Fig. 21. Error message when required data is missing

For algorithms of the fourth and fifth laboratory works, the graph type is additionally checked. If the user launches an algorithm requiring an undirected graph on a directed graph, the program reports that an undirected graph must be generated; see Fig. 22.

```
Enter choice:16


Error: Lab 5 requires an undirected graph.
Please generate an undirected graph (option 1, choose 2).
```

Fig. 22. Handling an incorrect graph type for the Eulerian algorithm

The correctness of vertex numbers and the admissibility of source and sink selection are also checked. With incorrect input, the program does not terminate abnormally; it prints a clear message and returns the user to the menu; see Fig. 23.

```
Enter choice:13


Error: No weight matrix generated. Use option 3 first.
```

Fig. 23. Handling incorrect user input

Conclusion

Based on the completed laboratory works, the following conclusions can be drawn:

In the first laboratory work, an algorithm for generating a random connected acyclic graph using the normal distribution was implemented. For the obtained graph, vertex eccentricities, radius, diameter, center, and diametral vertices were calculated. Shimbell's method for finding minimum and maximum paths of a specified length was also implemented, as well as an algorithm for determining all routes between two selected vertices.

In the second laboratory work, depth-first graph traversal was performed. For a weighted graph, the Floyd-Warshall algorithm was implemented; it finds shortest paths between all pairs of vertices and reconstructs the path between vertices selected by the user. The program also outputs the number of iterations, which makes it possible to estimate the complexity of the executed algorithms.

In the third laboratory work, capacity and cost matrices were generated based on a directed graph. For the constructed transport network, the maximum flow was found using the Ford-Fulkerson algorithm. After that, the minimum-cost flow was computed for a value equal to $\lfloor 2/3 \cdot \varphi_{\max} \rfloor$, using the previously implemented Floyd-Warshall algorithm to find shortest paths by cost.

In the fourth laboratory work, the number of spanning trees was found for an undirected graph using Kirchhoff's matrix-tree theorem. The minimum spanning tree was built using Boruvka's algorithm. The obtained spanning tree was encoded by the Prufer code and then decoded while preserving edge weights. The search for a minimum edge cover was also implemented both for the original graph and for the constructed spanning tree.

In the fifth laboratory work, the graph was checked for Eulerian property. If the graph was not Eulerian, the program modified it with output of the performed changes and then built an Eulerian cycle. For the even-numbered variant, the fundamental cutset system based on the minimum spanning tree was implemented, as well as obtaining other cutsets using the symmetric difference of selected fundamental cutsets.

The completion of the laboratory works made it possible to study and implement the main graph theory algorithms for directed and undirected graphs: traversals, shortest paths, flows, spanning trees, covers, tree coding, Eulerian cycles, and fundamental cutsets. All algorithms are combined in one console program with a menu, which allows the user to sequentially generate a graph, build the required matrices, and apply algorithms from different laboratory works.

An advantage of the program is its modular structure: common graph entities, distribution generation, and weight matrix generation are placed in separate files, while algorithms are divided by laboratory works. The use of STL containers, especially `vector`, simplifies work with matrices and edge lists. The program includes checks for user input and restrictions on launching algorithms, for example the requirement of a directed graph for flow algorithms and an undirected graph for spanning trees, Eulerian cycles, and cutsets.

The disadvantages include the absence of a graphical graph representation: results are printed to the console as matrices, edge lists, and textual algorithm logs. In addition, some algorithms do not have the most optimal asymptotic complexity for large graphs, for example the minimum-cost flow search through repeated application of the Floyd-Warshall algorithm. In the future, the program can be extended with graph visualization and more efficient implementations for large graphs.

The laboratory works were implemented in C++ in the CLion development environment.

References

[1] Novikov F. A. Discrete Mathematics for Programmers / F. A. Novikov. — 3rd ed. — St. Petersburg: Piter, 2009. — 364 p.

[2] Shaporev S. D. Discrete Mathematics / S. D. Shaporev. — St. Petersburg: BHV-Petersburg, 2010. — 400 p.

[3] Romanovsky I. V. Discrete Analysis / I. V. Romanovsky. — 3rd ed. — St. Petersburg: Nevsky Dialect; BHV-Petersburg, 2003. — 320 p.

[4] Haggarty R. Discrete Mathematics for Programmers / R. Haggarty; translated from English. — Moscow: Technosphere, 2005. — 326 p.

[5] Cormen T., Leiserson C., Rivest R., Stein C. Introduction to Algorithms / translated from English. — 3rd ed. — Moscow: Williams, 2013. — 1328 p.

[6] Asanov M. O., Baransky V. A., Rasin V. V. Discrete Mathematics: Graphs, Matroids, Algorithms / M. O. Asanov, V. A. Baransky, V. V. Rasin. — 2nd ed. — St. Petersburg: Lan, 2010. — 368 p.

[7] Vadzinsky R. N. Handbook of Probability Distributions / R. N. Vadzinsky. — St. Petersburg: Nauka, 2001. — 295 p.

[8] Erickson J. Algorithms / J. Erickson; translated from English. — Moscow: Williams, 2022. — 960 p.

[9] Section "Telematics" / electronic text. — URL: https://tema.spbstu.ru/tgraph/ (accessed: 20.05.2026)

Appendix A. Common Modules and the Combined Program

This appendix contains the complete source code of the common modules used by all laboratory works, as well as the code of the combined console program with a menu.

```
#pragma once

constexpr double SHIMBELL_INF  =  1e15;
constexpr double SHIMBELL_NINF = -1e15;
```

Listing 2. File constants.h

```
#pragma once
#include <vector>
#include <iostream>

class Graph {
public:
    int n;
    bool directed;
    std::vector<std::vector<int>> adj; // adj[i][j] == 1 if edge i->j exists

    Graph() : n(0), directed(false) {}
    Graph(int n, bool directed = false);

    void addEdge(int u, int v);
    bool hasEdge(int u, int v) const;

    // Returns out-neighbors (or all neighbors for undirected)
    std::vector<int> neighbors(int u) const;

    // Just the adjacency matrix (used right after generation)
    void printAdjMatrix() const;

    // Full view: adjacency list + edge list + adjacency matrix
    // weightMatrix may be nullptr - if provided it is printed as well
    void printFull(const std::vector<std::vector<double>>* weightMatrix = nullptr) const;

    bool isEmpty() const { return n == 0; }
};
```

Listing 3. File graph.h

```
#include "graph.h"
#include "constants.h"  // SHIMBELL_INF, SHIMBELL_NINF
#include <iomanip>
#include <string>

static inline int D(int v) { return v + 1; }

Graph::Graph(int n, bool directed)
    : n(n), directed(directed), adj(n, std::vector<int>(n, 0)) {}

void Graph::addEdge(int u, int v) {
    adj[u][v] = 1;
    if (!directed)
        adj[v][u] = 1;
}

```

```cpp
bool Graph::hasEdge(int u, int v) const {
    return adj[u][v] == 1;
}

std::vector<int> Graph::neighbors(int u) const {
    std::vector<int> result;
    for (int v = 0; v < n; v++)
        if (adj[u][v] == 1)
            result.push_back(v);
    return result;
}

// -- Just the adjacency matrix ------------------------------------------------
void Graph::printAdjMatrix() const {
    const std::string type = directed ? "DIRECTED" : "UNDIRECTED";
    const std::string sep(60, '-');

    std::cout << "\n  " << sep << "\n";
    std::cout << "  ADJACENCY MATRIX  [" << type << "]\n";
    std::cout << "  " << sep << "\n\n";

    const int CW = 5;
    std::cout << "       ";
    for (int j = 0; j < n; j++)
        std::cout << std::setw(CW) << D(j);
    std::cout << "\n  " << std::string(5 + n * CW, '-') << "\n";

    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << D(i) << " |";
        for (int j = 0; j < n; j++) {
            if (i == j)
                std::cout << std::setw(CW) << 0;
            else if (adj[i][j])
                std::cout << std::setw(CW) << 1;
            else
                std::cout << std::setw(CW) << "inf";
        }
        std::cout << "\n";
    }

    if (!directed)
        std::cout << "\n  Note: Undirected => matrix is symmetric (mat[i][j] == mat[j][i])\n
    ";
    std::cout << "\n  " << sep << "\n";
}

// -- Full view ----------------------------------------------------------------
void Graph::printFull(const std::vector<std::vector<double>>* W) const {
    const std::string type = directed ? "DIRECTED" : "UNDIRECTED";
    const std::string sep(60, '-');

    // -- Adjacency List ---------------------------------
    std::cout << "\n  " << sep << "\n";
    std::cout << "  ADJACENCY LIST  [" << type << "]\n";
    std::cout << "  " << sep << "\n\n";

    for (int u = 0; u < n; u++) {
        auto nb = neighbors(u);
        std::cout << "  Vertex " << std::setw(3) << D(u) << "  -->  ";
        if (nb.empty()) {
```

```
            std::cout << "(isolated)";
        } else {
            for (int i = 0; i < (int)nb.size(); i++) {
                if (i) std::cout << ", ";
                std::cout << D(nb[i]);
            }
        }
        std::cout << "   (degree=" << nb.size() << ")\n";
    }

    // -- Edge List ---------------------------------------------
    std::cout << "\n  EDGE LIST:\n";
    const std::string arrow = directed ? "  ---->  " : "  ----  ";
    for (int u = 0; u < n; u++)
        for (int v = directed ? 0 : u + 1; v < n; v++)
            if (adj[u][v])
                std::cout << "    " << std::setw(3) << D(u)
                          << arrow << std::setw(3) << D(v) << "\n";

    // -- Adjacency Matrix --------------------------------------
    std::cout << "\n  " << sep << "\n";
    std::cout << "  ADJACENCY MATRIX  [" << type << "]\n";
    std::cout << "  " << sep << "\n\n";

    const int ACW = 5;
    std::cout << "       ";
    for (int j = 0; j < n; j++)
        std::cout << std::setw(ACW) << D(j);
    std::cout << "\n  " << std::string(5 + n * ACW, '-') << "\n";

    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << D(i) << " |";
        for (int j = 0; j < n; j++) {
            if (i == j)
                std::cout << std::setw(ACW) << 0;
            else if (adj[i][j])
                std::cout << std::setw(ACW) << 1;
            else
                std::cout << std::setw(ACW) << "inf";
        }
        std::cout << "\n";
    }
    if (!directed)
        std::cout << "\n  Note: Undirected => matrix is symmetric (mat[i][j] == mat[j][i])\n
    ";

    // -- Weight Matrix (if available) --------------------------
    if (W) {
        std::cout << "\n  " << sep << "\n";
        std::cout << "  WEIGHT MATRIX  (INF = no edge, 0 = self)\n";
        std::cout << "  " << sep << "\n\n";

        const int CW = 8;
        std::cout << "         ";
        for (int j = 0; j < n; j++)
            std::cout << std::setw(CW) << D(j);
        std::cout << "\n  " << std::string(5 + n * CW, '-') << "\n";

        for (int i = 0; i < n; i++) {
            std::cout << "  " << std::setw(3) << D(i) << " |";
```

```
            for (int j = 0; j < n; j++) {
                double v = (*W)[i][j];
                if (v >= SHIMBELL_INF / 2)
                    std::cout << std::setw(CW) << "INF";
                else if (v <= SHIMBELL_NINF / 2)
                    std::cout << std::setw(CW) << "-INF";
                else
                    std::cout << std::setw(CW) << static_cast<long long>(v);
            }
            std::cout << "\n";
        }
    }

    std::cout << "\n  " << sep << "\n";
}
```

Listing 4. File graph.cpp

```
#pragma once
#include <vector>

// Generate one sample from N(0,1) using the Vadzinsky formula:
//   x = sqrt(12/n) * (sum_{i=1}^{n} r_i  -  n/2)
// where r_i ~ Uniform(0,1) and n is the number of uniform samples.
double normalSample(int n = 12);

// Generate a positive integer degree from the distribution.
// Returns a value in [1, maxDeg].
int sampleDegree(int maxDeg = 6);

// NEW: "tree" replaced with "graph" - this function is used for both the
//      undirected graph (where sum = 2*(n-1) matches a spanning tree) and
//      the directed DAG (where degrees are scaled further by EXTRA_EDGE_SCALE).
//      The constraint sum == 2*(numVertices-1) and all degrees >= 1 still holds.
std::vector<int> generateDegreeSequence(int numVertices);
```

Listing 5. File distribution.h

```
#include "distribution.h"
#include <cmath>
#include <random>
#include <algorithm>


static std::mt19937& getRng() {
    static std::mt19937 rng(std::random_device{}());
    return rng;
}

double normalSample(int n) {
    std::uniform_real_distribution<double> uniform(0.0, 1.0);
    auto& rng = getRng();
    double sum = 0.0;
    for (int i = 0; i < n; i++)
        sum += uniform(rng);
    // x = sqrt(12/n) * (sum - n/2)
    return std::sqrt(12.0 / n) * (sum - n / 2.0);
}

int sampleDegree(int maxDeg) {
    // Map N(0,1) to a positive integer in [1, maxDeg]
```

```
    double raw = std::abs(normalSample(12));
    // raw is half-normal; scale to [1, maxDeg]
    int d = 1 + static_cast<int>(raw * (maxDeg - 1) / 2.5);
    return std::clamp(d, 1, maxDeg);
}

std::vector<int> generateDegreeSequence(int numVertices) {//dui
    if (numVertices <= 1) return std::vector<int>(numVertices, 0);

    int target = 2 * (numVertices - 1);
    std::vector<int> degrees(numVertices);
    for (int i = 0; i < numVertices; i++)
        degrees[i] = sampleDegree(numVertices - 1);

    // Adjust sum to target by randomly incrementing/decrementing
    auto& rng = getRng();
    std::uniform_int_distribution<int> pick(0, numVertices - 1);

    int sum = 0;
    for (int d : degrees) sum += d;

    int iterations = 0;
    while (sum != target && iterations < 100000) {
        int v = pick(rng);
        if (sum > target && degrees[v] > 1) {
            degrees[v]--;
            sum--;
        } else if (sum < target && degrees[v] < numVertices - 1) {
            degrees[v]++;
            sum++;
        }
        iterations++;
    }

    return degrees;
}
```

Listing 6. File distribution.cpp

```
#pragma once
#include "graph.h"

// renamed from generateTree() to generateDAG() because the result is not
// a tree - a tree has exactly n-1 edges but our directed graph generates up to
// n*(n-1)/2 edges. The correct term is DAG (Directed Acyclic Graph).
// If directed == false, returns an undirected connected acyclic graph derived
// by symmetrizing the directed DAG (ignoring edge directions).
Graph generateDAG(int n, bool directed = false);
```

Listing 7. File dag_generator.h

```
#include "dag_generator.h"
#include "distribution.h"
#include <random>
#include <vector>
#include <numeric>
#include <algorithm>

static std::mt19937& getRng() {
    static std::mt19937 rng(std::random_device{}());
    return rng;
```

```
}

// ---------------------------------------------------------------------------
//  EXTRA_EDGE_SCALE: multiplier applied to the sampled out-degree for each
//  vertex in the directed DAG. Controls graph density.
//
//  1.0 = sparse    4.0 = medium    8.0 = dense (current)
// ---------------------------------------------------------------------------
// separate scales
static constexpr double EXTRA_EDGE_SCALE_DIRECTED   = 8.0;  // dense DAG
static constexpr double EXTRA_EDGE_SCALE_UNDIRECTED = 1.5;  // sparse enough to vary after
    symmetrize

static Graph buildDirectedDAG(int n, double scale);

//  Build an undirected graph from the same directed DAG model by ignoring
//  edge directions. This matches the lab wording: the undirected graph is
//  obtained from the generated oriented graph. Because the directed graph may
//  contain extra forward edges, the undirected version is connected and may
//  contain cycles, so MST algorithms are non-trivial.
// ---------------------------------------------------------------------------
static Graph buildUndirectedGraph(int n) {
    Graph dag = buildDirectedDAG(n, EXTRA_EDGE_SCALE_UNDIRECTED);
    Graph undirected(n, false);

    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
            if (dag.hasEdge(u, v))
                undirected.addEdge(u, v);

    return undirected;
}

// ---------------------------------------------------------------------------
//  DIRECTED: connected acyclic directed graph (DAG)
//  Algorithm - "generation by vertex degrees" as required by the lab:
//
//  1. Sample an OUT-DEGREE sequence from the Normal distribution via
//     generateDegreeSequence() - same function used for undirected.
//     Scale each degree by EXTRA_EDGE_SCALE for density; cap at available
//     forward targets so acyclicity is preserved.
//
//  2. Create a random topological ordering. Edges only go from lower to
//     higher position => acyclicity guaranteed by construction.
//
//  3. Spanning path topo[0]->topo[1]->...->topo[n-1] added first to
//     guarantee weak connectivity; counts against each vertex's degree budget.
//
//  4. Fill remaining out-degree budget per vertex with random forward edges.
// ---------------------------------------------------------------------------
static Graph buildDirectedDAG(int n, double scale) {
    Graph g(n, true);
    if (n <= 1) return g;
    if (n == 2) { g.addEdge(0, 1); return g; }

    auto& rng = getRng();

    // Step 1: random topological ordering
    std::vector<int> topo(n);
    std::iota(topo.begin(), topo.end(), 0);
```

```
    std::shuffle(topo.begin(), topo.end(), rng);

    // Step 2: sample out-degree for each vertex from Normal distribution,
    //         scaled by EXTRA_EDGE_SCALE, capped by available forward targets.
    std::vector<int> outDeg = generateDegreeSequence(n);
    for (int i = 0; i < n; i++) {
        int maxOut = n - i - 1;  // topo[i] can reach at most (n-i-1) vertices ahead
        outDeg[i] = std::min((int)std::round(outDeg[i] * scale), maxOut);
        outDeg[i] = std::max(outDeg[i], (i < n - 1) ? 1 : 0);  // at least 1 (except last)
    }

    // Step 3: spanning path for connectivity; consume one slot per vertex
    for (int i = 0; i < n - 1; i++) {
        g.addEdge(topo[i], topo[i + 1]);
        outDeg[i]--;
    }

    // Step 4: fill remaining degree budget with random forward edges
    for (int i = 0; i < n - 2; i++) {
        if (outDeg[i] <= 0) continue;

        std::vector<int> candidates;
        for (int j = i + 2; j < n; j++)
            candidates.push_back(j);
        std::shuffle(candidates.begin(), candidates.end(), rng);

        int added = 0;
        for (int j : candidates) {
            if (added >= outDeg[i]) break;
            int v = topo[j];
            if (!g.hasEdge(topo[i], v)) {
                g.addEdge(topo[i], v);
                added++;
            }
        }
    }

    return g;
}

// ---------------------------------------------------------------------------
//  Public entry point
//
//  renamed to generateDAG() - the result is a DAG, not a tree.
//  A tree has exactly n-1 edges; our directed graph has up to n*(n-1)/2 edges.
// ---------------------------------------------------------------------------
Graph generateDAG(int n, bool directed) {
    if (directed) return buildDirectedDAG(n, EXTRA_EDGE_SCALE_DIRECTED);
    else          return buildUndirectedGraph(n);
}
```

Listing 8. File dag_generator.cpp

```
#pragma once
#include "graph.h"
#include "constants.h"
#include "distribution.h"
#include <vector>

using Matrix = std::vector<std::vector<double>>;

```

```
enum WeightMode { POSITIVE, NEGATIVE, MIXED };

// Shared weight matrix generator used by all labs.
// Non-edges -> SHIMBELL_INF, diagonal -> SHIMBELL_INF (exact k-step semantics).
// Weights sampled from N(0,1)*5 rounded to integer, sign controlled by mode.
Matrix generateWeightMatrix(const Graph& g, WeightMode mode);
```

Listing 9. File weight_matrix.h

```
#include "weight_matrix.h"
#include <cmath>

static double sampleWeight(WeightMode mode) {
    double raw = normalSample(12) * 5.0;
    int w = static_cast<int>(std::round(raw));
    switch (mode) {
        case POSITIVE: return (w <= 0) ? 1 : w;
        case NEGATIVE: return (w >= 0) ? -1 : w;
        case MIXED:
        default:       return (w == 0) ? 1 : w;
    }
}

Matrix generateWeightMatrix(const Graph& g, WeightMode mode) {
    int n = g.n;
    Matrix W(n, std::vector<double>(n, SHIMBELL_INF));
    for (int i = 0; i < n; i++) {
        for (int j = g.directed ? 0 : i + 1; j < n; j++) {
            if (i == j) continue;
            if (g.hasEdge(i, j)) {
                double w = sampleWeight(mode);
                W[i][j] = w;
                if (!g.directed) W[j][i] = w;  // undirected edge has one weight
            }
        }
    }
    return W;
}
```

Listing 10. File weight_matrix.cpp

```
#include <iostream>
#include <limits>
#include <string>
#include <iomanip>
#include <cmath>
#include "graph.h"
#include "dag_generator.h"
#include "weight_matrix.h"
#include "eccentricity.h"
#include "shimbell.h"
#include "path_counter.h"
#include "dfs.h"
#include "floyd_warshall.h"
#include "ford_fulkerson.h"
#include "min_cost_flow.h"
#include "boruvka.h"
#include "edge_cover.h"
#include "kirchhoff.h"
#include "prufer.h"
#include "eulerian.h"
```

```cpp
#include "fundamental_cutsets.h"


// -- Global state ------------------------------------------------------------
static Graph  gGraph;
static bool   gGraphReady = false;

// Shared weight matrix (used by Lab 1 Shimbell, Lab 2 Floyd-Warshall, Lab 4 MST)
static Matrix gWeightMatrix;
static bool   gWeightReady = false;

// static std::vector<std::vector<double>> gFWWeightMatrix;  // removed: FW now uses gWeightMatrix directly
// static bool gFWWeightReady = false;

// Lab 3
static std::vector<std::vector<int>> gCapMatrix;
static std::vector<std::vector<int>> gCostMatrix;
static bool gFlowMatricesReady = false;

// Lab 4
static BoruvkaResult gMST;
static bool          gMSTReady   = false;
static PruferResult  gPrufer;
static bool          gPruferReady = false;

static inline int D(int v) { return v + 1; }

static void clearInput() {
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
}
static int readInt(const std::string& prompt) {
    int v;
    while (true) {
        std::cout << prompt;
        if (std::cin >> v) { clearInput(); return v; }
        std::cout << "  Invalid input. Please enter an integer.\n";
        clearInput();
    }
}
static void requireGraph() {
    std::cout << "  Error: No graph generated yet. Use option 1 first.\n";
}
static void requireWeight() {
    std::cout << "  Error: No weight matrix generated. Use option 3 first.\n";
}
static void requireFlowMatrices() {
    std::cout << "  Error: No capacity/cost matrices. Use option 9 first.\n";
}
static void requireMST() {
    std::cout << "  Error: No MST computed. Use option 13 (Boruvka) first.\n";
}
static bool requireLab4Undirected() {
    if (!gGraphReady) {
        requireGraph();
        return false;
    }
    if (gGraph.directed) {
        std::cout << "  Error: Lab 4 requires an undirected graph.\n"
```

```
                  << "  Please generate an undirected graph (option 1, choose 2).\n";
        return false;
    }
    return true;
}
static bool requireLab5Undirected() {
    if (!gGraphReady) {
        requireGraph();
        return false;
    }
    if (gGraph.directed) {
        std::cout << "  Error: Lab 5 requires an undirected graph.\n"
                  << "  Please generate an undirected graph (option 1, choose 2).\n";
        return false;
    }
    return true;
}

// -- 1. Generate graph ------------------------------------------------------
void menuGenerateGraph() {
    int n = readInt("  Number of vertices: ");
    if (n <= 0) { std::cout << "  Must be > 0.\n"; return; }
    std::cout << "  Graph type:\n    1. Directed\n    2. Undirected\n";
    bool directed = (readInt("  Choose [1/2]: ") == 1);
    gGraph = generateDAG(n, directed);
    gGraphReady        = true;
    gWeightReady       = false;
    gFlowMatricesReady = false;
    gMSTReady          = false;
    gPruferReady       = false;
    gGraph.printAdjMatrix();
    std::cout << "  Graph with " << n << " vertices generated";
    if (!directed) std::cout << " (undirected)";
    std::cout << ".\n  Generate weight matrix with option 3.\n";
}

// -- 3. Generate weight matrix (shared) -------------------------------------
void menuGenerateWeightMatrix() {
    if (!gGraphReady) { requireGraph(); return; }
    std::cout << "  Weight mode:\n    1. Positive only\n    2. Negative only\n    3. Mixed\n
    ";
    int wChoice = readInt("  Choose [1/2/3]: ");
    WeightMode mode;
    switch (wChoice) {
        case 2:  mode = NEGATIVE; break;
        case 3:  mode = MIXED;    break;
        default: mode = POSITIVE; break;
    }
    gWeightMatrix = generateWeightMatrix(gGraph, mode);
    gWeightReady  = true;
    gMSTReady     = false;
    gPruferReady  = false;
    std::cout << "  Weight matrix generated.\n";
    // print it
    const int W = 8, n = gGraph.n;
    std::cout << "\n       ";
    for (int j = 0; j < n; j++) std::cout << std::setw(W) << D(j);
    std::cout << "\n  " << std::string(5 + n * W, '-') << "\n";
    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << D(i) << " |";
```

```
        for (int j = 0; j < n; j++) {
            double v = gWeightMatrix[i][j];
            if      (v >= SHIMBELL_INF / 2)  std::cout << std::setw(W) << "INF";
            else if (v <= SHIMBELL_NINF / 2) std::cout << std::setw(W) << "-INF";
            else    std::cout << std::setw(W) << static_cast<long long>(v);
        }
        std::cout << "\n";
    }
}

// -- Shimbell pretty-print ------------------------------------------------
static void printShimbellMatrix(const Matrix& M) {
    int n = static_cast<int>(M.size());
    const int W = 8;
    std::cout << "       ";
    for (int j = 0; j < n; j++) std::cout << std::setw(W) << D(j);
    std::cout << "\n  " << std::string(5 + n * W, '-') << "\n";
    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << D(i) << " |";
        for (int j = 0; j < n; j++) {
            if      (M[i][j] >= SHIMBELL_INF / 2)  std::cout << std::setw(W) << "INF";
            else if (M[i][j] <= SHIMBELL_NINF / 2) std::cout << std::setw(W) << "-INF";
            else    std::cout << std::setw(W) << static_cast<long long>(M[i][j]);
        }
        std::cout << "\n";
    }
}

// -- 4. Eccentricities ----------------------------------------------------
void menuEccentricity() {
    if (!gGraphReady) { requireGraph(); return; }
    auto m = computeMetrics(gGraph);
    printMetrics(m);
}

// -- 5. Shimbell ----------------------------------------------------------
void menuShimbell() {
    if (!gGraphReady) { requireGraph(); return; }
    if (!gWeightReady) { requireWeight(); return; }
    const std::string sep(60, '-');
    std::cout << "\n  " << sep << "\n  WEIGHT MATRIX\n  " << sep << "\n\n";
    printShimbellMatrix(gWeightMatrix);
    int k = readInt("\n  Enter k (number of steps): ");
    if (k < 0) { std::cout << "  k must be >= 0.\n"; return; }
    auto [minMat, maxMat] = shimbell(gWeightMatrix, k);
    std::cout << "\n  " << sep << "\n  SHIMBELL MIN-path (k=" << k << ")\n  " << sep << "\n\
    n";
    printShimbellMatrix(minMat);
    std::cout << "\n  " << sep << "\n  SHIMBELL MAX-path (k=" << k << ")\n  " << sep << "\n\
    n";
    printShimbellMatrix(maxMat);
}

// -- 6. Routes ------------------------------------------------------------
void menuPaths() {
    if (!gGraphReady) { requireGraph(); return; }
    if (!gWeightReady) { requireWeight(); return; }
    gGraph.printFull(&gWeightMatrix);
    std::cout << "  Vertices are numbered 1 to " << gGraph.n << ".\n";
    int src = readInt("  Source vertex: ")      - 1;
```

```
    int dst = readInt("  Destination vertex: ") - 1;
    if (src < 0 || src >= gGraph.n || dst < 0 || dst >= gGraph.n) {
        std::cout << "  Vertex out of range.\n"; return;
    }
    auto paths = findAllPaths(gGraph, src, dst);
    printPaths(paths, src, dst);
}

// -- 7. DFS ------------------------------------------------------------------
void menuDFS() {
    if (!gGraphReady) { requireGraph(); return; }
    int start = readInt("  Enter start vertex: ") - 1;
    if (start < 0 || start >= gGraph.n) { std::cout << "  Vertex out of range.\n"; return; }
    dfs(gGraph, start);
}

// -- 8. Floyd-Warshall -------------------------------------------------------
void menuFloydWarshall() {
    if (!gGraphReady)  { requireGraph();  return; }
    if (!gWeightReady) { requireWeight(); return; }

    // Build FW matrix from shared weight matrix: Shimbell has INF on diagonal,
    // Floyd-Warshall needs 0 there.
    const int n = gGraph.n;
    std::vector<std::vector<double>> fw(gWeightMatrix);
    for (int i = 0; i < n; i++) fw[i][i] = 0.0;

    int src = readInt("  Source vertex: ") - 1;
    int dst = readInt("  Destination vertex: ") - 1;
    if (src < 0 || src >= n || dst < 0 || dst >= n) {
        std::cout << "  Vertex out of range.\n"; return;
    }
    FloydResult res = floydWarshall(gGraph, fw);
    printFloydResult(res, fw, n, src, dst);
}

// -- 9. Generate capacity and cost matrices (Lab 3) --------------------------
void menuGenerateFlowMatrices() {
    if (!gGraphReady) { requireGraph(); return; }
    if (!gGraph.directed) {
        std::cout << "  Error: Lab 3 requires a directed graph.\n"
                  << "  Please generate a directed graph (option 1, choose 1).\n";
        return;
    }
    gCapMatrix  = generateCapacityMatrix(gGraph);
    gCostMatrix = generateCostMatrix(gGraph);
    gFlowMatricesReady = true;
    const int n = gGraph.n;
    printCapacityMatrix(gCapMatrix, n);
    printCostMatrix(gCostMatrix, n);
    // Show out-degree / in-degree to help pick source and sink
    std::cout << "\n  Vertex degrees (to help choose source / sink):\n";
    std::cout << "    Vertex | Out-degree | In-degree\n";
    std::cout << "    " << std::string(34, '-') << "\n";
    for (int i = 0; i < n; i++) {
        int out = 0, in = 0;
        for (int j = 0; j < n; j++) {
            if (gCapMatrix[i][j] > 0) out++;
            if (gCapMatrix[j][i] > 0) in++;
        }
```

```
        std::cout << "      " << std::setw(3) << i+1
                  << "   |    " << std::setw(3) << out
                  << "     |   " << std::setw(3) << in << "\n";
    }
    std::cout << "  Good source: high out-degree, low in-degree.\n";
    std::cout << "  Good sink:   high in-degree,  low out-degree.\n";
}

// -- 10. Ford-Fulkerson ------------------------------------------------------
void menuFordFulkerson() {
    if (!gGraphReady)        { requireGraph();        return; }
    if (!gFlowMatricesReady) { requireFlowMatrices(); return; }
    const int n = gGraph.n;
    int src = readInt("  Source vertex: ") - 1;
    int dst = readInt("  Sink vertex:   ") - 1;
    if (src < 0 || src >= n || dst < 0 || dst >= n) {
        std::cout << "  Vertex out of range.\n"; return;
    }
    if (src == dst) {
        std::cout << "  Source and sink must be different vertices.\n"; return;
    }
    FFResult res = fordFulkerson(n, gCapMatrix, src, dst);
    if (res.maxFlow == 0)
        std::cout << "  No path exists from vertex " << src+1 << " to vertex " << dst+1
                  << ".\n  Max flow = 0. Choose a different source/sink pair.\n"
                  << "  Hint: pick a vertex with only outgoing edges as source,\n"
                  << "        and a vertex with only incoming edges as sink.\n";
    else
        printFFResult(res, gCapMatrix, n);
}

// -- 11. Min-cost flow -------------------------------------------------------
void menuMinCostFlow() {
    if (!gGraphReady)        { requireGraph();        return; }
    if (!gFlowMatricesReady) { requireFlowMatrices(); return; }
    const int n = gGraph.n;
    int src = readInt("  Source vertex: ") - 1;
    int dst = readInt("  Sink vertex:   ") - 1;
    if (src < 0 || src >= n || dst < 0 || dst >= n) {
        std::cout << "  Vertex out of range.\n"; return;
    }
    if (src == dst) {
        std::cout << "  Source and sink must be different vertices.\n"; return;
    }
    MCFResult res = minCostFlow(n, gCapMatrix, gCostMatrix, src, dst);
    printMCFResult(res, gCapMatrix, gCostMatrix, n);
}

// -- 12. Kirchhoff (spanning tree count) -------------------------------------
void menuKirchhoff() {
    if (!requireLab4Undirected()) { return; }
    long long count = countSpanningTrees(gGraph);
    printKirchhoffResult(gGraph, count);
}

// -- 13. Boruvka (MST) -------------------------------------------------------
void menuBoruvka() {
    if (!requireLab4Undirected()) { return; }
    if (!gWeightReady) { requireWeight(); return; }
    gMST = boruvka(gGraph, gWeightMatrix);
```

```
317     gMSTReady    = true;
318     gPruferReady = false;
319     printBoruvkaResult(gMST);
320 }
321 
322 // -- 14. Edge cover ------------------------------------------------------
323 void menuEdgeCover() {
324     if (!requireLab4Undirected()) { return; }
325     if (!gWeightReady) { requireWeight(); return; }
326     std::cout << "  Work on:\n    1. MST (requires Boruvka, option 13)\n    2. Full graph\n"
    ;
327     bool useMST = (readInt("  Choose [1/2]: ") == 1);
328     if (useMST && !gMSTReady) { requireMST(); return; }
329     EdgeCoverResult res = minEdgeCover(gGraph, gWeightMatrix, useMST,
330                                        useMST ? gMST.edges : std::vector<MSTEdge>{});
331     printEdgeCoverResult(res);
332 }
333 
334 // -- 15. Prufer encode / decode ------------------------------------------
335 void menuPrufer() {
336     if (!requireLab4Undirected()) { return; }
337     if (!gMSTReady) { requireMST(); return; }
338     if ((int)gMST.edges.size() != gGraph.n - 1) {
339         std::cout << "  Error: Prüfer coding requires a spanning tree with n-1 edges.\n"
340                   << "  Re-run Boruvka on a connected undirected weighted graph.\n";
341         return;
342     }
343     gPrufer = pruferEncode(gMST.edges, gGraph.n);
344     gPruferReady = true;
345     printPruferResult(gPrufer, gGraph.n);
346     auto decoded = pruferDecode(gPrufer.code, gPrufer.weights, gGraph.n);
347     printDecodedTree(decoded);
348 }
349 
350 
351 // -- 16. Eulerian graph / Euler cycle ------------------------------------
352 void menuEulerianCycle() {
353     if (!requireLab5Undirected()) { return; }
354     EulerianResult res = buildEulerianCycle(gGraph);
355     printEulerianResult(res);
356 }
357 
358 // -- 17. Fundamental cutsets (variant 36 is even) ------------------------
359 void menuFundamentalCutsets() {
360     if (!requireLab5Undirected()) { return; }
361     if (!gWeightReady) { requireWeight(); return; }
362 
363     std::cout << "  Building MST with Boruvka (Lab 4 algorithm)...\n";
364     gMST = boruvka(gGraph, gWeightMatrix);
365     gMSTReady = true;
366     gPruferReady = false;
367     printBoruvkaResult(gMST);
368 
369     if ((int)gMST.edges.size() != gGraph.n - 1) {
370         std::cout << "  Error: fundamental cutsets require a spanning tree.\n";
371         return;
372     }
373 
374     auto cutsets = buildFundamentalCutsets(gGraph, gMST.edges);
375     printFundamentalCutsets(cutsets);
```

```
376     if (cutsets.empty()) {
377         return;
378     }
379 
380     int count = readInt("  Number of cutsets for symmetric difference: ");
381     if (count < 0) {
382         std::cout << "  Count must be >= 0.\n";
383         return;
384     }
385 
386     std::vector<int> selected;
387     for (int i = 0; i < count; i++) {
388         int idx = readInt("  Cutset number: ");
389         if (idx < 1 || idx > (int)cutsets.size()) {
390             std::cout << "  Ignoring invalid cutset number " << idx << ".\n";
391             continue;
392         }
393         selected.push_back(idx);
394     }
395     printCutsetSymmetricDifference(cutsets, selected);
396 }
397 
398 // -- Menu ------------------------------------------------------------------
399 void printMenu() {
400     std::cout << "\n========================================\n"
401               << "   GRAPH THEORY LABS\n"
402               << "========================================\n"
403               << "  1. Generate graph\n"
404               << "  2. Show graph (adjacency list / matrix)\n"
405               << "  3. Generate weight matrix\n"
406               << "  =====Lab 1=====\n"
407               << "  4. Eccentricities, center, diametral vertices\n"
408               << "  5. Shimbell's method (min/max paths, k steps)\n"
409               << "  6. Find all routes between two vertices\n"
410               << "  =====Lab 2=====\n"
411               << "  7. DFS traversal\n"
412               << "  8. Shortest path: Floyd-Warshall\n"
413               << "  =====Lab 3=====\n"
414               << "  9.  Generate capacity and cost matrices\n"
415               << "  10. Max flow: Ford-Fulkerson\n"
416               << "  11. Min-cost flow\n"
417               << "  =====Lab 4=====\n"
418               << "  12. Kirchhoff: count spanning trees\n"
419               << "  13. Boruvka: minimum spanning tree\n"
420               << "  14. Minimum edge cover\n"
421               << "  15. Prufer encode / decode\n"
422               << "  =====Lab 5=====\n"
423               << "  16. Eulerian check / modify / Euler cycle\n"
424               << "  17. Fundamental cutsets from MST\n"
425               << "  0.  Exit\n"
426               << "----------------------------------------\n";
427 }
428 
429 int main() {
430     int choice = -1;
431     while (choice != 0) {
432         printMenu();
433         choice = readInt("  Enter choice: ");
434         std::cout << "\n";
435         switch (choice) {
```

```
            case 1:  menuGenerateGraph();    break;
            case 2:
                if (gGraphReady)
                    gGraph.printFull(gWeightReady ? &gWeightMatrix : nullptr);
                else requireGraph();
                break;
            case 3:  menuGenerateWeightMatrix(); break;
            case 4:  menuEccentricity();         break;
            case 5:  menuShimbell();             break;
            case 6:  menuPaths();                break;
            case 7:  menuDFS();                  break;
            case 8:  menuFloydWarshall();        break;
            case 9:  menuGenerateFlowMatrices(); break;
            case 10: menuFordFulkerson();        break;
            case 11: menuMinCostFlow();          break;
            case 12: menuKirchhoff();            break;
            case 13: menuBoruvka();              break;
            case 14: menuEdgeCover();            break;
            case 15: menuPrufer();               break;
            case 16: menuEulerianCycle();        break;
            case 17: menuFundamentalCutsets();   break;
            case 0:  std::cout << "  Goodbye.\n"; break;
            default: std::cout << "  Unknown option.\n"; break;
        }
    }
    return 0;
}
```

Listing 11. File main.cpp of the combined program

Appendix B. Source Code of Laboratory Work No. 1

```h
#pragma once
#include "graph.h"
#include <vector>

// BFS from src following directed (or undirected) edges.
// Returns dist[v] = shortest-path distance, or -1 if unreachable.
std::vector<int> bfs(const Graph& g, int src);

struct GraphMetrics {
    std::vector<int> eccentricities;        // ecc[v]; 0 if no other vertex is reachable
    std::vector<std::vector<int>> distMatrix; // distMatrix[u][v] = BFS distance, -1 if
    unreachable
    int radius;                              // min eccentricity among vertices with ecc > 0
    int diameter;                            // max eccentricity
    std::vector<int> center;                 // vertices achieving the radius
    std::vector<int> diametralVertices;      // vertices achieving the diameter
};

GraphMetrics computeMetrics(const Graph& g);
void printMetrics(const GraphMetrics& m);
```

Listing 12. File eccentricity.h

```cpp
#include "eccentricity.h"
#include <queue>
#include <algorithm>
#include <iostream>
#include <iomanip>

static inline int D(int v) { return v + 1; }

std::vector<int> bfs(const Graph& g, int src) {
    std::vector<int> dist(g.n, -1);
    std::queue<int> q;
    dist[src] = 0;
    q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g.neighbors(u)) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist;
}

GraphMetrics computeMetrics(const Graph& g) {
    GraphMetrics m;
    m.eccentricities.resize(g.n, 0);
    m.distMatrix.resize(g.n);

    for (int v = 0; v < g.n; v++) {
        m.distMatrix[v] = bfs(g, v);
        int maxDist = 0;
```

```
        for (int u = 0; u < g.n; u++) {
            if (u != v && m.distMatrix[v][u] != -1)
                maxDist = std::max(maxDist, m.distMatrix[v][u]);
        }
        m.eccentricities[v] = maxDist;
    }

    m.diameter = *std::max_element(m.eccentricities.begin(), m.eccentricities.end());

    m.radius = m.diameter;
    for (int v = 0; v < g.n; v++)
        if (m.eccentricities[v] > 0)
            m.radius = std::min(m.radius, m.eccentricities[v]);
    if (m.diameter == 0) m.radius = 0;

    for (int v = 0; v < g.n; v++) {
        if (m.eccentricities[v] == m.radius && m.radius > 0)
            m.center.push_back(v);
        if (m.eccentricities[v] == m.diameter && m.diameter > 0)
            m.diametralVertices.push_back(v);
    }

    return m;
}

void printMetrics(const GraphMetrics& m) {
    int n = static_cast<int>(m.eccentricities.size());

    // -- Distance matrix ------------------------------------
    std::cout << "\n";
    std::cout << "       ";
    for (int j = 0; j < n; j++)
        std::cout << std::setw(5) << D(j);
    std::cout << "\n  " << std::string(5 + n * 5, '-') << "\n";

    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << D(i) << " |";
        for (int j = 0; j < n; j++) {
            int d = m.distMatrix[i][j];
            if (d == -1)
                std::cout << std::setw(5) << "inf";
            else
                std::cout << std::setw(5) << d;
        }
        std::cout << "\n";
    }

    // -- Eccentricity table ---------------------------------
    std::cout << "\n";
    std::cout << "  Vertex  | Eccentricity | Role\n";
    std::cout << "  --------+--------------+-------------------------\n";
    for (int v = 0; v < n; v++) {
        int e = m.eccentricities[v];
        std::string role = "";
        if (m.diameter > 0 && e == m.diameter) role = "DIAMETRICAL";
        if (m.radius  > 0 && e == m.radius)   {
            role = (role.empty() ? "" : role + " + ");
            role += "CENTER";
        }
        if (role.empty()) role = "-";
```

```
        std::cout << "  " << std::setw(6) << D(v)
                  << "  |  " << std::setw(8) << e
                  << "    | " << role << "\n";
    }

    // -- Summary ------------------------------------------------
    std::cout << "\n  Radius   = " << m.radius   << "\n";
    std::cout << "  Diameter = " << m.diameter << "\n";

    std::cout << "\n  CENTER vertices      = { ";
    for (int v : m.center) std::cout << D(v) << " ";
    std::cout << "}\n";

    std::cout << "  DIAMETRICAL vertices = { ";
    for (int v : m.diametralVertices) std::cout << D(v) << " ";
    std::cout << "}\n";
}
```

Listing 13. File eccentricity.cpp

```
#pragma once
#include "graph.h"
#include "constants.h"
#include "weight_matrix.h"  // Matrix, WeightMode, generateWeightMatrix (shared)
#include <vector>

// Shimbell's algorithm.
//   k == 0  -> identity matrix (diagonal 0, off-diagonal +INF / -INF)
//   k >= 1  -> multiply W by itself k times in the (min/max, +) tropical semiring
//
// Returns {minMatrix, maxMatrix}.
std::pair<Matrix, Matrix> shimbell(const Matrix& W, int k);

// Pretty-print a matrix.  Values >= INF_THRESHOLD are shown as "INF",
// values <= -INF_THRESHOLD are shown as "-INF".
void printMatrix(const Matrix& M);
```

Listing 14. File shimbell.h

```
#include "shimbell.h"
// #include "distribution.h"  // no longer needed here - moved to shared/weight_matrix.cpp
// #include <cmath>           // no longer needed here
#include <algorithm>
#include <iostream>
#include <iomanip>


// -- Tropical matrix multiply -------------------------------------------------

static Matrix tropicalMin(const Matrix& A, const Matrix& B) {
    int n = static_cast<int>(A.size());
    Matrix C(n, std::vector<double>(n, SHIMBELL_INF));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++) {
            if (A[i][k] >= SHIMBELL_INF) continue;
            for (int j = 0; j < n; j++) {
                if (B[k][j] >= SHIMBELL_INF) continue;
                double v = A[i][k] + B[k][j];
                if (v < C[i][j]) C[i][j] = v;
            }
        }
```

```
    return C;
}

static Matrix tropicalMax(const Matrix& A, const Matrix& B) {
    int n = static_cast<int>(A.size());
    Matrix C(n, std::vector<double>(n, SHIMBELL_NINF));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++) {
            if (A[i][k] <= SHIMBELL_NINF) continue;
            for (int j = 0; j < n; j++) {
                if (B[k][j] <= SHIMBELL_NINF) continue;
                double v = A[i][k] + B[k][j];
                if (v > C[i][j]) C[i][j] = v;
            }
        }
    return C;
}

// -- Identity matrices ------------------------------------------------------

static Matrix identityMin(int n) {
    Matrix I(n, std::vector<double>(n, SHIMBELL_INF));
    for (int i = 0; i < n; i++) I[i][i] = 0.0;
    return I;
}

static Matrix identityMax(int n) {
    Matrix I(n, std::vector<double>(n, SHIMBELL_NINF));
    for (int i = 0; i < n; i++) I[i][i] = 0.0;
    return I;
}

// Build the "max" version of the weight matrix:
// edges keep their weight, non-edges → SHIMBELL_NINF, diagonal → SHIMBELL_NINF.
// W[i][i] == SHIMBELL_INF, so the condition below correctly skips the diagonal.
static Matrix toMaxMatrix(const Matrix& W) {
    int n = static_cast<int>(W.size());
    Matrix M(n, std::vector<double>(n, SHIMBELL_NINF));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            if (W[i][j] < SHIMBELL_INF)   // real edges only (diagonal is INF -> skipped)
                M[i][j] = W[i][j];
        }
    return M;
}

// -- Public shimbell() ------------------------------------------------------

std::pair<Matrix, Matrix> shimbell(const Matrix& W, int k) {
    int n = static_cast<int>(W.size());

    if (k == 0)
        return { identityMin(n), identityMax(n) };

    Matrix Wmax = toMaxMatrix(W);

    Matrix minResult = W;
    Matrix maxResult = Wmax;

    for (int step = 1; step < k; step++) {
```

```
        minResult = tropicalMin(minResult, W);
        maxResult = tropicalMax(maxResult, Wmax);
    }

    return { minResult, maxResult };
}

// -- Print -------------------------------------------------------------------

void printMatrix(const Matrix& M) {
    int n = static_cast<int>(M.size());
    const int W = 8;

    std::cout << "    ";
    for (int j = 0; j < n; j++)
        std::cout << std::setw(W) << j;
    std::cout << "\n    ";
    for (int j = 0; j < n; j++)
        std::cout << std::string(W, '-');
    std::cout << "\n";

    for (int i = 0; i < n; i++) {
        std::cout << std::setw(2) << i << " |";
        for (int j = 0; j < n; j++) {
            if (M[i][j] >= SHIMBELL_INF / 2)
                std::cout << std::setw(W) << "INF";
            else if (M[i][j] <= SHIMBELL_NINF / 2)
                std::cout << std::setw(W) << "-INF";
            else {
                std::cout << std::setw(W) << static_cast<long long>(M[i][j]);
            }
        }
        std::cout << "\n";
    }
}
```

Listing 15. File shimbell.cpp

```
#pragma once
#include "graph.h"
#include <vector>

// Returns all simple paths from src to dst as lists of internal (0-based) indices.
// If src == dst returns one empty path.
std::vector<std::vector<int>> findAllPaths(const Graph& g, int src, int dst);

// Convenience wrappers
bool pathExists(const Graph& g, int src, int dst);
int  countPaths (const Graph& g, int src, int dst);

// Pretty-print: "Route 1: { 1 -> 3 -> 5 }"  (1-indexed display)
void printPaths(const std::vector<std::vector<int>>& paths, int src, int dst);
```

Listing 16. File path_counter.h

```
#include "path_counter.h"
#include <iostream>
#include <iomanip>

static inline int D(int v) { return v + 1; }

```

```cpp
// -- DFS that collects full paths ---------------------------------------------
static void dfs(const Graph& g, int u, int dst,
                std::vector<bool>& visited,
                std::vector<int>& current,
                std::vector<std::vector<int>>& result) {
    if (u == dst) {
        result.push_back(current);
        return;
    }
    visited[u] = true;
    for (int v : g.neighbors(u)) {
        if (!visited[v]) {
            current.push_back(v);
            dfs(g, v, dst, visited, current, result);
            current.pop_back();
        }
    }
    visited[u] = false;
}

std::vector<std::vector<int>> findAllPaths(const Graph& g, int src, int dst) {
    std::vector<std::vector<int>> result;
    if (src < 0 || src >= g.n || dst < 0 || dst >= g.n)
        return result;
    if (src == dst) {
        result.push_back({src});
        return result;
    }
    std::vector<bool> visited(g.n, false);
    std::vector<int>  current = {src};
    dfs(g, src, dst, visited, current, result);
    return result;
}

bool pathExists(const Graph& g, int src, int dst) {
    return !findAllPaths(g, src, dst).empty();
}

int countPaths(const Graph& g, int src, int dst) {
    return static_cast<int>(findAllPaths(g, src, dst).size());
}

void printPaths(const std::vector<std::vector<int>>& paths, int src, int dst) {
    std::cout << "\n  Path from " << D(src) << " to " << D(dst) << ": ";
    if (paths.empty()) {
        std::cout << "DOES NOT EXIST\n";
        return;
    }
    std::cout << "EXISTS  (" << paths.size() << " route"
              << (paths.size() == 1 ? "" : "s") << ")\n\n";
    std::cout << "  Routes:\n";
    for (int i = 0; i < (int)paths.size(); i++) {
        std::cout << "    Route " << i + 1 << ":  { ";
        for (int j = 0; j < (int)paths[i].size(); j++) {
            if (j) std::cout << " -> ";
            std::cout << D(paths[i][j]);
        }
        std::cout << " }\n";
    }
```

```
66 }
```

Listing 17. File path_counter.cpp

Appendix C. Source Code of Laboratory Work No. 2

```
#pragma once
#include "graph.h"

// Performs DFS traversal from 'start'.
// Prints traversal order, discovery order, and total iteration count.
void dfs(const Graph& g, int start);
```

Listing 18. File dfs.h

```
#include "dfs.h"
#include <iostream>
#include <stack>
#include <vector>
#include <iomanip>

// Iterative DFS using an explicit stack.
// Counts every "iteration" - each time we pop a vertex and inspect its neighbors.
void dfs(const Graph& g, int start) {
    const std::string sep(60, '-');
    std::cout << "\n  " << sep << "\n";
    std::cout << "  DFS TRAVERSAL  (start = vertex " << start + 1 << ")\n";
    std::cout << "  " << sep << "\n\n";

    std::vector<bool> visited(g.n, false);
    std::vector<int>  order;
    std::stack<int>   stk;
    long long         iterations = 0;

    stk.push(start);

    while (!stk.empty()) {
        int u = stk.top();
        stk.pop();
        iterations++;

        if (visited[u]) continue;
        visited[u] = true;
        order.push_back(u);
        iterations++;

        // Push neighbors in reverse order so smaller-index neighbors are
        // visited first (matches recursive DFS order).
        auto nb = g.neighbors(u);
        for (int i = (int)nb.size() - 1; i >= 0; i--) {
            iterations++;
            if (!visited[nb[i]])
                stk.push(nb[i]);
        }
    }

    // -- Traversal order ------------------------------------
    std::cout << "  Traversal order:\n  ";
    for (int i = 0; i < (int)order.size(); i++) {
        if (i) std::cout << " -> ";
        std::cout << order[i] + 1;
    }
```

```
    std::cout << "\n\n";

    // -- Visited / unreachable --------------------------------
    std::vector<int> unvisited;
    for (int v = 0; v < g.n; v++)
        if (!visited[v]) unvisited.push_back(v);

    std::cout << "  Vertices visited   : " << order.size() << " / " << g.n << "\n";
    if (!unvisited.empty()) {
        std::cout << "  Unreachable        : ";
        for (int i = 0; i < (int)unvisited.size(); i++) {
            if (i) std::cout << ", ";
            std::cout << unvisited[i] + 1;
        }
        std::cout << "\n";
        std::cout << "  (Graph not fully reachable from vertex " << start + 1 << ")\n";
    }

    std::cout << "\n  Total iterations   : " << iterations << "\n";
    std::cout << "\n  " << sep << "\n";
}
```

Listing 19. File dfs.cpp

```
#pragma once
#include "graph.h"
#include "weight_matrix.h"  // Matrix, WeightMode, generateWeightMatrix (shared)
#include <vector>


// Result bundle returned by floydWarshall()
struct FloydResult {
    std::vector<std::vector<double>> T;   // distance matrix
    std::vector<std::vector<int>>    H;   // path matrix (1-based next hop, 0 = no path)
    long long iterations;                 // always n^3 (unconditionally counted)
    bool hasNegCycle;
    std::vector<int> negCycleVerts;       // 0-based indices of vertices with T[j][j]<0
};

// Runs Floyd-Warshall
//   loop order i,j,k  |  condition j≠i & T[j][i]≠INF & i≠k & T[i][k]≠INF
//   & (T[j][k]=INF or T[j][k] > T[j][i]+T[i][k])
//   iterations counted unconditionally → always n^3
FloydResult floydWarshall(const Graph&                                g,
                          const std::vector<std::vector<double>>& W);

// Prints C, T, H matrices + path reconstruction + iteration count
void printFloydResult(const FloydResult&                      res,
                      const std::vector<std::vector<double>>& W,
                      int n, int src, int dst);
```

Listing 20. File floyd_warshall.h

```
#include "floyd_warshall.h"
#include "constants.h"
#include <iostream>
#include <iomanip>
// #include <random>    // no longer needed here - moved to shared/weight_matrix.cpp
#include <vector>
#include <algorithm>

```

```cpp

// ------------------------------------------------------------------------
//  Print helpers
// ------------------------------------------------------------------------
static void printDMatrix(const std::vector<std::vector<double>>& M,
                         int n, const std::string& title) {
    const int CW = 8;
    std::cout << "\n  " << title << "\n\n";
    std::cout << "       ";
    for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
    std::cout << "\n  " << std::string(7 + n * CW, '-') << "\n";
    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << i + 1 << " |";
        for (int j = 0; j < n; j++) {
            double v = M[i][j];
            if      (v >= SHIMBELL_INF / 2)  std::cout << std::setw(CW) << "INF";
            else if (v <= SHIMBELL_NINF / 2) std::cout << std::setw(CW) << "-INF";
            else    std::cout << std::setw(CW) << static_cast<long long>(v);
        }
        std::cout << "\n";
    }
}

static void printHMatrix(const std::vector<std::vector<int>>& H,
                         int n, const std::string& title) {
    const int CW = 8;
    std::cout << "\n  " << title << "\n\n";
    std::cout << "       ";
    for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
    std::cout << "\n  " << std::string(7 + n * CW, '-') << "\n";
    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << i + 1 << " |";
        for (int j = 0; j < n; j++) std::cout << std::setw(CW) << H[i][j];
        std::cout << "\n";
    }
}

// ------------------------------------------------------------------------
//  Floyd-Warshall
//
//  Loop order:  i (intermediate vertex), j (source row), k (destination col)
//  Condition:   j≠i  &  T[j][i]≠INF  &  i≠k  &  T[i][k]≠INF
//               &  (T[j][k]=INF  OR  T[j][k] > T[j][i]+T[i][k])
//  Iterations:  counted unconditionally inside innermost loop → always n^3
//  H matrix:    H[i][j] = j+1 (1-based) if direct edge, 0 if none
//               updated as H[j][k] := H[j][i]  (first hop on new path)
//  Neg-cycle:   after main loop, if T[j][j] < 0 → no solution
// ------------------------------------------------------------------------
FloydResult floydWarshall(const Graph&                                g,
                          const std::vector<std::vector<double>>& W) {
    const int n = g.n;

    // -- Initialize T and H ---------------------------------
    std::vector<std::vector<double>> T = W;
    std::vector<std::vector<int>>    H(n, std::vector<int>(n, 0));

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            H[i][j] = (W[i][j] < SHIMBELL_INF / 2) ? (j + 1) : 0;

```

```
    // -- Triple loop - order: i, j, k ------------------------
    long long iterations = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                iterations++;          // unconditional - always n^3 total

                if (j == i) continue;//same ver
                if (T[j][i] >= SHIMBELL_INF / 2) continue;//no path
                if (i == k) continue;
                if (T[i][k] >= SHIMBELL_INF / 2) continue;

                double via = T[j][i] + T[i][k];
                if (T[j][k] >= SHIMBELL_INF / 2 || T[j][k] > via) {//gocha
                    H[j][k] = H[j][i];   // first hop on new shorter path
                    T[j][k] = via;
                }
            }
        }
    }

    // -- Negative-cycle detection ----------------------------
    bool hasNegCycle = false;
    std::vector<int> negCycleVerts;
    for (int j = 0; j < n; j++) {
        if (T[j][j] < -1e-9) {
            hasNegCycle = true;
            negCycleVerts.push_back(j);
        }
    }

    return { T, H, iterations, hasNegCycle, negCycleVerts };
}

// ---------------------------------------------------------------------------
//  Print full result
// ---------------------------------------------------------------------------
void printFloydResult(const FloydResult&                        res,
                      const std::vector<std::vector<double>>& W,
                      int n, int src, int dst) {
    const std::string sep(60, '-');

    std::cout << "\n  " << sep << "\n";
    std::cout << "  FLOYD-WARSHALL  (src=" << src + 1
              << "  dst=" << dst + 1 << ")\n";
    std::cout << "  " << sep << "\n";

    printDMatrix(W,     n, "C = WEIGHT MATRIX  (INF = no edge, 0 = self):");
    printDMatrix(res.T, n, "T = DISTANCE MATRIX  (all-pairs shortest paths):");
    printHMatrix(res.H, n, "H = PATH MATRIX  (H[i][j] = first next hop, 0 = no path):");

    // Negative cycle
    if (res.hasNegCycle) {
        //std::cout << "\n  *** NEGATIVE CYCLE DETECTED - no solution ***\n";
        //std::cout << "  Vertices on negative diagonal: ";
        for (int i = 0; i < (int)res.negCycleVerts.size(); i++) {
            if (i) std::cout << ", ";
            std::cout << res.negCycleVerts[i] + 1;
        }
        std::cout << "\n";
```

```
        std::cout << "\n  Total iterations: " << res.iterations
                  << "  (n=" << n << "  where  n^3=" << (long long)n*n*n << ")\n";
        std::cout << "\n  " << sep << "\n";
        return;
    }

    // Path reconstruction
    //   w := src;  yield w
    //   while w != dst do  w := H[w][dst];  yield w
    std::cout << "\n  Shortest path from " << src + 1
              << " to " << dst + 1 << ":\n";

    if (src == dst) {
        std::cout << "  Path: " << src + 1 << "  (same vertex, distance = 0)\n";
    } else if (res.T[src][dst] >= SHIMBELL_INF / 2) {
        std::cout << "  No path exists.\n";
    } else {
        std::vector<int> path;
        int w = src;
        int guard = n + 2;
        bool ok = true;
        path.push_back(w);
        while (w != dst) {
            int nxt = res.H[w][dst] - 1;  // H is 1-based to convert to 0-based
            if (nxt < 0 || nxt >= n || --guard < 0) { ok = false; break; }
            w = nxt;
            path.push_back(w);
        }
        if (!ok) {
            std::cout << "  (reconstruction failed)\n";
        } else {
            std::cout << "  Path    : ";
            for (int i = 0; i < (int)path.size(); i++) {
                if (i) std::cout << " -> ";
                std::cout << path[i] + 1;
            }
            std::cout << "\n";
            std::cout << "  Distance: "
                      << static_cast<long long>(res.T[src][dst]) << "\n";
        }
    }

    std::cout << "\n  Total iterations: " << res.iterations
              << "  (n=" << n << "  where  n^3=" << (long long)n*n*n << ")\n";
    std::cout << "\n  " << sep << "\n";
}
```

Listing 21. File floyd_warshall.cpp

Appendix D. Source Code of Laboratory Work No. 3

```
#pragma once
#include "graph.h"
#include <vector>
#include <climits>

// Capacity matrix: cap[i][j] in [1,20] for edge (i,j), else 0
std::vector<std::vector<int>> generateCapacityMatrix(const Graph& g);

// Cost matrix: D[i][j] in [1,10] for edge (i,j), else 0
std::vector<std::vector<int>> generateCostMatrix(const Graph& g);

struct FFResult {
    std::vector<std::vector<int>> flow;  // net flow on each arc
    int maxFlow;
    int src, dst;
};

// BFS-based Ford-Fulkerson (Edmonds-Karp), prints each augmenting path
FFResult fordFulkerson(int n,
                       const std::vector<std::vector<int>>& cap,
                       int src, int dst);

// Same but silent (no output), used internally by min-cost flow
FFResult fordFulkersonSilent(int n,
                             const std::vector<std::vector<int>>& cap,
                             int src, int dst);

void printCapacityMatrix(const std::vector<std::vector<int>>& cap, int n);
void printCostMatrix(const std::vector<std::vector<int>>& cost, int n);
void printFFResult(const FFResult& res,
                   const std::vector<std::vector<int>>& cap, int n);
```

Listing 22. File ford_fulkerson.h

```
#include "ford_fulkerson.h"
#include <iostream>
#include <iomanip>
#include <random>
#include <queue>
#include <algorithm>
#include <vector>

static std::mt19937 rng3(std::random_device{}());

std::vector<std::vector<int>> generateCapacityMatrix(const Graph& g) {
    const int n = g.n;
    std::vector<std::vector<int>> cap(n, std::vector<int>(n, 0));
    std::uniform_int_distribution<int> dist(1, 20);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
            if (g.hasEdge(u, v))
                cap[u][v] = dist(rng3);
    return cap;
}

std::vector<std::vector<int>> generateCostMatrix(const Graph& g) {
```

```
    const int n = g.n;
    std::vector<std::vector<int>> cost(n, std::vector<int>(n, 0));
    std::uniform_int_distribution<int> dist(1, 10);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
            if (g.hasEdge(u, v))
                cost[u][v] = dist(rng3);
    return cost;
}

// BFS finds an augmenting path; prev[v] = predecessor of v on path
static bool bfs(const std::vector<std::vector<int>>& resCap, int n,
                int src, int dst, std::vector<int>& prev) {
    prev.assign(n, -1);
    prev[src] = src;
    std::queue<int> q;
    q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v = 0; v < n; v++) {
            if (prev[v] == -1 && resCap[u][v] > 0) {
                prev[v] = u;
                if (v == dst) return true;
                q.push(v);
            }
        }
    }
    return false;
}

static FFResult runFF(int n, const std::vector<std::vector<int>>& cap,
                      int src, int dst, bool verbose) {
    std::vector<std::vector<int>> resCap = cap;
    std::vector<std::vector<int>> flow(n, std::vector<int>(n, 0));
    int maxFlow = 0;
    int pathNum = 0;

    std::vector<int> prev;
    while (bfs(resCap, n, src, dst, prev)) {
        // Bottleneck
        int delta = INT_MAX;
        for (int v = dst; v != src; v = prev[v])
            delta = std::min(delta, resCap[prev[v]][v]);

        // Augment flow along path
        for (int v = dst; v != src; v = prev[v]) {
            int u = prev[v];
            resCap[u][v] -= delta;
            resCap[v][u] += delta;
            flow[u][v] += delta;
            flow[v][u] -= delta;
        }
        maxFlow += delta;
        pathNum++;

        if (verbose) {
            // Reconstruct path for display
            std::vector<int> path;
            for (int v = dst; v != src; v = prev[v]) path.push_back(v);
            path.push_back(src);
```

```
            std::reverse(path.begin(), path.end());
            std::cout << "  Path " << pathNum << ": ";
            for (int i = 0; i < (int)path.size(); i++) {
                if (i) std::cout << " -> ";
                std::cout << path[i] + 1;
            }
            std::cout << "  (delta = " << delta << ")\n";
        }
    }
    return { flow, maxFlow, src, dst };
}

FFResult fordFulkerson(int n, const std::vector<std::vector<int>>& cap,
                       int src, int dst) {
    return runFF(n, cap, src, dst, true);
}

FFResult fordFulkersonSilent(int n, const std::vector<std::vector<int>>& cap,
                             int src, int dst) {
    return runFF(n, cap, src, dst, false);
}

static void printIntMatrix(const std::vector<std::vector<int>>& M, int n,
                           const std::string& title) {
    const int CW = 6;
    std::cout << "\n  " << title << "\n\n";
    std::cout << "       ";
    for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
    std::cout << "\n  " << std::string(7 + n * CW, '-') << "\n";
    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << i + 1 << " |";
        for (int j = 0; j < n; j++)
            std::cout << std::setw(CW) << M[i][j];
        std::cout << "\n";
    }
}

void printCapacityMatrix(const std::vector<std::vector<int>>& cap, int n) {
    printIntMatrix(cap, n, "CAPACITY MATRIX  Omega[i][j]:");
}

void printCostMatrix(const std::vector<std::vector<int>>& cost, int n) {
    printIntMatrix(cost, n, "COST MATRIX  D[i][j]:");
}

void printFFResult(const FFResult& res,
                   const std::vector<std::vector<int>>& cap, int n) {
    const std::string sep(60, '-');
    std::cout << "\n  " << sep << "\n";
    std::cout << "  FORD-FULKERSON RESULT\n";
    std::cout << "  Source: " << res.src + 1 << "   Sink: " << res.dst + 1 << "\n";
    std::cout << "  Maximum flow  phi_max = " << res.maxFlow << "\n";
    std::cout << "  " << sep << "\n";
    printCapacityMatrix(cap, n);

    // Print only arcs that carry positive flow
    std::cout << "\n  FLOW on each arc (only arcs with flow > 0):\n";
    bool any = false;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
```

```
            if (res.flow[i][j] > 0) {
                std::cout << "    (" << i + 1 << " -> " << j + 1 << ")  "
                          << "flow=" << res.flow[i][j]
                          << "  cap=" << cap[i][j] << "\n";
                any = true;
            }
    if (!any) std::cout << "    (none)\n";
    std::cout << "\n  " << sep << "\n";
}
```

Listing 23. File ford_fulkerson.cpp

```
#pragma once
#include "ford_fulkerson.h"
#include <vector>

struct MCFResult {
    std::vector<std::vector<int>> flow;  // net flow on each arc
    int theta;                           // target flow = floor(2/3 * maxFlow)
    int totalFlow;                       // flow achieved (== theta if feasible)
    long long totalCost;
    int maxFlow;
};

// Successive shortest paths (Bellman-Ford) to find min-cost flow of theta units
// theta = floor(2/3 * maxFlow)
MCFResult minCostFlow(int n,
                      const std::vector<std::vector<int>>& cap,
                      const std::vector<std::vector<int>>& cost,
                      int src, int dst);

void printMCFResult(const MCFResult& res,
                    const std::vector<std::vector<int>>& cap,
                    const std::vector<std::vector<int>>& cost, int n);
```

Listing 24. File min_cost_flow.h

```
#include "min_cost_flow.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <algorithm>
#include <climits>

// ---------------------------------------------------------------------------
//  Floyd-Warshall on the residual cost graph.
//
//  Used at every iteration of the successive-shortest-paths algorithm
//  (lab task: "use the previously implemented Floyd-Warshall").
//
//  Builds T (distance) and H (next-hop, 1-based) matrices from:
//    resCap[u][v] > 0  →  arc exists with cost resCost[u][v]
//    resCap[u][v] == 0 →  arc absent
//
//  Loop order: i (intermediate), j (source row), k (destination col).
//  H[j][k] = first hop from j toward k (1-based vertex index).
// ---------------------------------------------------------------------------
struct FWCostResult {
    std::vector<std::vector<int>> T;   // shortest cost distances
    std::vector<std::vector<int>> H;   // next-hop matrix (1-based, 0 = no path)
};
```

```
25
26 static const int FW_INF = INT_MAX / 2;
27
28 static FWCostResult floydWarshallCost(
29         const std::vector<std::vector<int>>& resCap,
30         const std::vector<std::vector<int>>& resCost,
31         int n) {
32
33     std::vector<std::vector<int>> T(n, std::vector<int>(n, FW_INF));
34     std::vector<std::vector<int>> H(n, std::vector<int>(n, 0));
35
36     // Initialise: diagonal = 0, edges = cost where capacity > 0
37     for (int i = 0; i < n; i++) {
38         T[i][i] = 0;
39         for (int j = 0; j < n; j++) {
40             if (i != j && resCap[i][j] > 0) {
41                 T[i][j] = resCost[i][j];
42                 H[i][j] = j + 1;          // direct edge → first hop is j
43             }
44         }
45     }
46
47     // Triple loop - same order as Lab 2: i (intermediate), j (row), k (col)
48     for (int i = 0; i < n; i++) {
49         for (int j = 0; j < n; j++) {
50             if (j == i || T[j][i] == FW_INF) continue;
51             for (int k = 0; k < n; k++) {
52                 if (i == k || T[i][k] == FW_INF) continue;
53                 int via = T[j][i] + T[i][k];
54                 if (T[j][k] == FW_INF || T[j][k] > via) {
55                     T[j][k] = via;
56                     H[j][k] = H[j][i];    // first hop toward k is same as toward i
57                 }
58             }
59         }
60     }
61
62     return { T, H };
63 }
64
65 // Reconstruct path from src to dst using Floyd-Warshall next-hop matrix H.
66 // Returns empty vector if no path.
67 static std::vector<int> reconstructPath(
68         const std::vector<std::vector<int>>& H,
69         int src, int dst, int n) {
70
71     if (H[src][dst] == 0) return {};
72     std::vector<int> path;
73     int w = src;
74     path.push_back(w);
75     int guard = n + 2;
76     while (w != dst) {
77         int nxt = H[w][dst] - 1;          // convert 1-based → 0-based
78         if (nxt < 0 || nxt >= n || --guard < 0) return {};
79         w = nxt;
80         path.push_back(w);
81     }
82     return path;
83 }
84
```

```cpp
// ---------------------------------------------------------------------------
//  Min-cost flow  -  successive shortest paths via Floyd-Warshall
//
//  Algorithm :
//   1. phi_max = Ford-Fulkerson(cap, src, dst).
//      theta = floor(2/3 * phi_max).
//   2. Build residual network (initially same as cap/cost).
//   3. While total_flow < theta:
//      a. Run Floyd-Warshall on residual cost graph.
//      b. Get shortest path src→dst from H matrix.
//      c. delta = min(residual caps on path, theta - total_flow).
//      d. Augment flow by delta along path.
//      e. Update residual network (lecture page 4 rules):
//           - Forward arc (u,v): resCap[u][v] -= delta
//           - Reverse arc (v,u): resCap[v][u] += delta,
//                                resCost[v][u] = -d(u,v)  (negated original cost)
// ---------------------------------------------------------------------------
MCFResult minCostFlow(int n,
                      const std::vector<std::vector<int>>& cap,
                      const std::vector<std::vector<int>>& cost,
                      int src, int dst) {

    // Step 1: max flow
    FFResult ff = fordFulkersonSilent(n, cap, src, dst);
    int maxFlow = ff.maxFlow;
    int theta   = (2 * maxFlow) / 3;

    std::cout << "  phi_max = " << maxFlow
              << "   theta = floor(2/3 * " << maxFlow << ") = " << theta << "\n";

    if (maxFlow == 0) {
        std::cout << "  No path from source to sink - max flow is 0.\n";
        return { std::vector<std::vector<int>>(n, std::vector<int>(n,0)),
                 0, 0, 0LL, 0 };
    }
    if (theta == 0) {
        std::cout << "  theta = 0, nothing to route.\n";
        return { std::vector<std::vector<int>>(n, std::vector<int>(n,0)),
                 0, 0, 0LL, maxFlow };
    }

    // Step 2: residual network starts as the original capacity / cost matrices
    std::vector<std::vector<int>> resCap  = cap;
    std::vector<std::vector<int>> resCost = cost;
    std::vector<std::vector<int>> flow(n, std::vector<int>(n, 0));

    int       totalFlow = 0;
    long long totalCost = 0;
    int       pathNum   = 0;

    // Step 3: successive shortest paths
    while (totalFlow < theta) {

        // 3a. Floyd-Warshall on current residual cost graph
        FWCostResult fw = floydWarshallCost(resCap, resCost, n);

        // 3b. shortest path src→dst
        if (fw.T[src][dst] == FW_INF) break;   // no path in residual graph
        std::vector<int> path = reconstructPath(fw.H, src, dst, n);
        if (path.empty()) break;
```

```
145
146         // 3c. bottleneck = min(residual cap on path, remaining demand)
147         int delta = theta - totalFlow;
148         for (int i = 0; i + 1 < (int)path.size(); i++)
149             delta = std::min(delta, resCap[path[i]][path[i+1]]);
150
151         // 3d-e. augment and update residual network
152         for (int i = 0; i + 1 < (int)path.size(); i++) {
153             int u = path[i], v = path[i+1];
154             int usedCost = resCost[u][v];
155             resCap[u][v]  -= delta;              // forward arc: reduce capacity
156             resCap[v][u]  += delta;              // reverse arc: add capacity
157             resCost[v][u]  = -usedCost;          // reverse arc cancels traversed edge
158             flow[u][v]    += delta;
159             flow[v][u]    -= delta;
160         }
161         totalFlow += delta;
162         totalCost += (long long)delta * fw.T[src][dst];
163         pathNum++;
164
165         // Print this step
166         std::cout << "  Step " << pathNum << " [Floyd-Warshall]: ";
167         for (int i = 0; i < (int)path.size(); i++) {
168             if (i) std::cout << " -> ";
169             std::cout << path[i] + 1;
170         }
171         std::cout << "  delta=" << delta
172                   << "  path_cost=" << fw.T[src][dst]
173                   << "  total_flow=" << totalFlow << "\n";
174     }
175
176     return { flow, theta, totalFlow, totalCost, maxFlow };
177 }
178
179 // ---------------------------------------------------------------------------
180 //  Print result
181 // ---------------------------------------------------------------------------
182 void printMCFResult(const MCFResult& res,
183                     const std::vector<std::vector<int>>& cap,
184                     const std::vector<std::vector<int>>& cost, int n) {
185     const std::string sep(60, '-');
186     std::cout << "\n  " << sep << "\n";
187     std::cout << "  MIN-COST FLOW RESULT\n";
188     std::cout << "  phi_max        = " << res.maxFlow   << "\n";
189     std::cout << "  theta (2/3*max)= " << res.theta     << "\n";
190     std::cout << "  Flow achieved  = " << res.totalFlow << "\n";
191     std::cout << "  Total cost     = " << res.totalCost << "\n";
192     std::cout << "  " << sep << "\n";
193
194     std::cout << "\n  FLOW DETAILS (arcs with flow > 0):\n";
195     std::cout << "    Arc       flow   cap   cost/unit   cost\n";
196     std::cout << "    " << std::string(52, '-') << "\n";
197     long long checkCost = 0;
198     bool any = false;
199     for (int i = 0; i < n; i++)
200         for (int j = 0; j < n; j++)
201             if (res.flow[i][j] > 0) {
202                 long long c = (long long)cost[i][j] * res.flow[i][j];
203                 checkCost += c;
204                 std::cout << "    (" << i+1 << "->" << j+1 << ")"
```

```
                          << std::setw(8) << res.flow[i][j]
                          << std::setw(6) << cap[i][j]
                          << std::setw(12) << cost[i][j]
                          << std::setw(8) << c << "\n";
                any = true;
            }
    if (!any) std::cout << "    (none)\n";
    std::cout << "    " << std::string(52, '-') << "\n";
    std::cout << "    Total cost = " << checkCost << "\n";
    std::cout << "\n  " << sep << "\n";
}
```

Listing 25. File min_cost_flow.cpp

Appendix E. Source Code of Laboratory Work No. 4

```
#pragma once
#include "graph.h"

// Kirchhoff's Matrix-Tree Theorem:
//   Build Laplacian L where L[i][i]=degree(i), L[i][j]=-1 if edge(i,j)
//   Remove any row k and column k; the determinant of the remaining
//   (n-1)x(n-1) matrix equals the number of spanning trees.
//   (Graph treated as undirected regardless of directed flag.)
long long countSpanningTrees(const Graph& g);

void printKirchhoffResult(const Graph& g, long long count);
```

Listing 26. File kirchhoff.h

```
#include "kirchhoff.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <cmath>

// Build the Kirchhoff (Laplacian) matrix treating the graph as undirected.
// L[i][j] = -1 if edge {i,j} exists (in either direction), L[i][i] = degree(i).
static std::vector<std::vector<double>> buildLaplacian(const Graph& g) {
    const int n = g.n;
    std::vector<std::vector<double>> L(n, std::vector<double>(n, 0.0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j) continue;
            if (g.hasEdge(i, j) || g.hasEdge(j, i)) {
                L[i][j] = -1.0;
                L[i][i] += 1.0;
            }
        }
    }
    return L;
}

// Gaussian elimination determinant with partial pivoting.
static double gaussianDet(std::vector<std::vector<double>> M) {
    int n = (int)M.size();
    double det = 1.0;
    for (int col = 0; col < n; col++) {
        int pivot = -1;
        double best = 0.0;
        for (int row = col; row < n; row++) {
            if (std::abs(M[row][col]) > best) {
                best = std::abs(M[row][col]);
                pivot = row;
            }
        }
        if (pivot == -1 || best < 1e-9) return 0.0;
        if (pivot != col) {
            std::swap(M[pivot], M[col]);
            det = -det;
        }
        det *= M[col][col];
```

```
        double inv = 1.0 / M[col][col];
        for (int row = col + 1; row < n; row++) {
            double factor = M[row][col] * inv;
            for (int k = col; k < n; k++)
                M[row][k] -= factor * M[col][k];
        }
    }
    return det;
}

long long countSpanningTrees(const Graph& g) {
    int n = g.n;
    if (n < 2) return 0;

    auto L = buildLaplacian(g);

    // Cofactor A_{00}: remove row 0 and col 0, compute det of (n-1)x(n-1) submatrix
    std::vector<std::vector<double>> sub(n - 1, std::vector<double>(n - 1));
    for (int i = 1; i < n; i++)
        for (int j = 1; j < n; j++)
            sub[i - 1][j - 1] = L[i][j];

    double d = gaussianDet(sub);
    return static_cast<long long>(std::round(d));
}

void printKirchhoffResult(const Graph& g, long long count) {
    const int n = g.n;
    const std::string sep(60, '-');

    std::cout << "\n  " << sep << "\n";
    std::cout << "  KIRCHHOFF'S MATRIX-TREE THEOREM\n";
    std::cout << "  " << sep << "\n";

    auto L = buildLaplacian(g);

    const int CW = 5;
    std::cout << "\n  Kirchhoff (Laplacian) matrix B:\n";
    std::cout << "  B[i][i] = degree(i),  B[i][j] = -1 if edge {i,j}, else 0\n\n";
    std::cout << "       ";
    for (int j = 0; j < n; j++) std::cout << std::setw(CW) << j + 1;
    std::cout << "\n  " << std::string(7 + n * CW, '-') << "\n";
    for (int i = 0; i < n; i++) {
        std::cout << "  " << std::setw(3) << i + 1 << " |";
        for (int j = 0; j < n; j++)
            std::cout << std::setw(CW) << (int)L[i][j];
        std::cout << "\n";
    }

    // Print cofactor submatrix (remove row 1, col 1)
    std::cout << "\n  Cofactor A_11 - delete row 1 and col 1:\n\n";
    std::cout << "       ";
    for (int j = 1; j < n; j++) std::cout << std::setw(CW) << j + 1;
    std::cout << "\n  " << std::string(7 + (n - 1) * CW, '-') << "\n";
    for (int i = 1; i < n; i++) {
        std::cout << "  " << std::setw(3) << i + 1 << " |";
        for (int j = 1; j < n; j++)
            std::cout << std::setw(CW) << (int)L[i][j];
        std::cout << "\n";
    }
```

```
103
104     std::cout << "\n  det(A_11) = " << count << "\n";
105     std::cout << "  Number of spanning trees = " << count << "\n";
106     std::cout << "\n  " << sep << "\n";
107 }
```

Listing 27. File kirchhoff.cpp

```
1  #pragma once
2  #include "graph.h"
3  #include <vector>
4
5  struct MSTEdge {
6      int u, v;
7      double weight;
8  };
9
10 struct BoruvkaResult {
11     std::vector<MSTEdge> edges;  // MST edges (n-1 edges)
12     double totalWeight;
13 };
14
15 // Boruvka's algorithm (lecture tg9.pdf pp.62-66):
16 //   While more than 1 component: for each component find the cheapest
17 //   outgoing edge (safe edge); add all safe edges; merge components.
18 //   Tie-breaking: smallest edge index (u*n+v).
19 //   Graph treated as undirected. Weight matrix W[i][j] gives edge weights.
20 BoruvkaResult boruvka(const Graph& g,
21                       const std::vector<std::vector<double>>& W);
22
23 void printBoruvkaResult(const BoruvkaResult& res);
```

Listing 28. File boruvka.h

```
1  #include "boruvka.h"
2  #include "constants.h"
3  #include <iostream>
4  #include <iomanip>
5  #include <vector>
6  #include <map>
7  #include <numeric>
8  #include <functional>
9
10 BoruvkaResult boruvka(const Graph& g,
11                       const std::vector<std::vector<double>>& W) {
12     const int n = g.n;
13
14     // Local Union-Find
15     std::vector<int> parent(n), rnk(n, 0);
16     std::iota(parent.begin(), parent.end(), 0);
17
18     std::function<int(int)> find = [&](int x) -> int {
19         if (parent[x] != x) parent[x] = find(parent[x]);
20         return parent[x];
21     };
22     auto unite = [&](int a, int b) -> bool {
23         a = find(a); b = find(b);
24         if (a == b) return false;
25         if (rnk[a] < rnk[b]) std::swap(a, b);
26         parent[b] = a;
27         if (rnk[a] == rnk[b]) rnk[a]++;
```

```
        return true;
    };

    // Print current component partition
    auto printComps = [&]() {
        std::map<int, std::vector<int>> comps;
        for (int i = 0; i < n; i++) comps[find(i)].push_back(i);
        std::cout << "  Components:";
        for (auto& [root, members] : comps) {
            std::cout << " {";
            for (int k = 0; k < (int)members.size(); k++) {
                if (k) std::cout << ",";
                std::cout << members[k] + 1;
            }
            std::cout << "}";
        }
        std::cout << "\n";
    };

    std::vector<MSTEdge> mst;
    int round = 0;

    while ((int)mst.size() < n - 1) {
        round++;
        std::cout << "  --- Round " << round << " ---\n";
        printComps();

        // For each component: find cheapest outgoing edge
        std::vector<int>    bestU(n, -1), bestV(n, -1);
        std::vector<double> bestW(n, SHIMBELL_INF);

        for (int u = 0; u < n; u++) {
            for (int v = u + 1; v < n; v++) {
                bool edge = g.hasEdge(u, v) || g.hasEdge(v, u);
                if (!edge) continue;
                // For undirected, W[u][v] == W[v][u] after the weight fix.
                // Take whichever direction is valid.
                double w = (W[u][v] < SHIMBELL_INF / 2) ? W[u][v]
                         : (W[v][u] < SHIMBELL_INF / 2 ? W[v][u] : SHIMBELL_INF);
                if (w >= SHIMBELL_INF / 2) continue;

                int cu = find(u), cv = find(v);
                if (cu == cv) continue;

                // Update best for component of u
                if (w < bestW[cu] ||
                    (w == bestW[cu] && u * n + v < bestU[cu] * n + bestV[cu])) {
                    bestW[cu] = w; bestU[cu] = u; bestV[cu] = v;
                }
                // Update best for component of v
                if (w < bestW[cv] ||
                    (w == bestW[cv] && u * n + v < bestU[cv] * n + bestV[cv])) {
                    bestW[cv] = w; bestU[cv] = u; bestV[cv] = v;
                }
            }
        }

        // Add all safe edges
        bool added = false;
        for (int c = 0; c < n; c++) {
```

```
            if (bestU[c] == -1) continue;
            int u = bestU[c], v = bestV[c];
            if (unite(u, v)) {
                double w = bestW[c];
                mst.push_back({ u, v, w });
                std::cout << "    Add edge (" << u + 1 << ", " << v + 1
                          << ")  weight=" << w << "\n";
                added = true;
            }
        }
        if (!added) {
            std::cout << "  Warning: no safe edges found - graph may be disconnected.\n";
            break;
        }
    }

    double total = 0.0;
    for (auto& e : mst) total += e.weight;
    return { mst, total };
}

void printBoruvkaResult(const BoruvkaResult& res) {
    const std::string sep(60, '-');
    std::cout << "\n  " << sep << "\n";
    std::cout << "  BORUVKA MST RESULT\n";
    std::cout << "  " << sep << "\n";
    std::cout << "\n  MST edges:\n";
    std::cout << "    Arc        weight\n";
    std::cout << "    " << std::string(22, '-') << "\n";
    for (auto& e : res.edges)
        std::cout << "    (" << e.u + 1 << " -> " << e.v + 1
                  << ")    " << e.weight << "\n";
    std::cout << "\n  Total MST weight = " << res.totalWeight << "\n";
    std::cout << "\n  " << sep << "\n";
}
```

Listing 29. File boruvka.cpp

```
#pragma once
#include "graph.h"
#include "boruvka.h"
#include <vector>

struct EdgeCoverResult {
    std::vector<MSTEdge> cover;   // edges in the minimum edge cover
    int matchingSize;             // size of maximum matching used
};

// Minimum edge cover via:
//   1. Find maximum matching M (Edmonds blossom, general undirected graph)
//   2. For each unmatched vertex add any incident edge
//   |min cover| = n - |M| for graphs without isolated vertices
//
// useMST: if true, work on mstEdges (a tree, which is always bipartite).
//         if false, work on the full undirected graph with weight matrix W.
EdgeCoverResult minEdgeCover(const Graph& g,
                             const std::vector<std::vector<double>>& W,
                             bool useMST,
                             const std::vector<MSTEdge>& mstEdges);

```

```
void printEdgeCoverResult(const EdgeCoverResult& res);
```

Listing 30. File edge_cover.h

```
#include "edge_cover.h"
#include "constants.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <queue>
#include <algorithm>

// Edmonds blossom maximum matching for a general undirected graph.
// Minimum edge cover size is n - |maximum matching| for graphs without isolated
// vertices, so this keeps the full-graph case correct even when it is not a tree.
class GeneralMatching {
public:
    GeneralMatching(const std::vector<std::vector<int>>& adj, int n)
        : adj(adj), n(n), match(n, -1), parent(n), base(n),
          used(n), blossom(n) {}

    int run(std::vector<int>& outMatch) {
        int result = 0;
        for (int root = 0; root < n; root++) {
            if (match[root] != -1) continue;
            int endpoint = findAugmentingPath(root);
            if (endpoint == -1) continue;

            while (endpoint != -1) {
                int pv = parent[endpoint];
                int next = match[pv];
                match[endpoint] = pv;
                match[pv] = endpoint;
                endpoint = next;
            }
            result++;
        }
        outMatch = match;
        return result;
    }

private:
    const std::vector<std::vector<int>>& adj;
    int n;
    std::vector<int> match;
    std::vector<int> parent;
    std::vector<int> base;
    std::vector<bool> used;
    std::vector<bool> blossom;
    std::queue<int> q;

    int lca(int a, int b) {
        std::vector<bool> seen(n, false);
        while (true) {
            a = base[a];
            seen[a] = true;
            if (match[a] == -1) break;
            a = parent[match[a]];
        }
        while (true) {
            b = base[b];
```

```
            if (seen[b]) return b;
            b = parent[match[b]];
        }
    }

    void markPath(int v, int b, int child) {
        while (base[v] != b) {
            blossom[base[v]] = true;
            blossom[base[match[v]]] = true;
            parent[v] = child;
            child = match[v];
            v = parent[match[v]];
        }
    }

    int findAugmentingPath(int root) {
        std::fill(used.begin(), used.end(), false);
        std::fill(parent.begin(), parent.end(), -1);
        for (int i = 0; i < n; i++) base[i] = i;

        q = std::queue<int>();
        q.push(root);
        used[root] = true;

        while (!q.empty()) {
            int v = q.front();
            q.pop();

            for (int u = 0; u < n; u++) {
                if (!adj[v][u] || base[v] == base[u] || match[v] == u) continue;

                if (u == root || (match[u] != -1 && parent[match[u]] != -1)) {
                    int curBase = lca(v, u);
                    std::fill(blossom.begin(), blossom.end(), false);
                    markPath(v, curBase, u);
                    markPath(u, curBase, v);

                    for (int i = 0; i < n; i++) {
                        if (!blossom[base[i]]) continue;
                        base[i] = curBase;
                        if (!used[i]) {
                            used[i] = true;
                            q.push(i);
                        }
                    }
                } else if (parent[u] == -1) {
                    parent[u] = v;
                    if (match[u] == -1)
                        return u;

                    int next = match[u];
                    used[next] = true;
                    q.push(next);
                }
            }
        }

        return -1;
    }
};
```

```
118
119 static int maxMatching(const std::vector<std::vector<int>>& adj, int n,
120                        std::vector<int>& match) {
121     return GeneralMatching(adj, n).run(match);
122 }
123
124 EdgeCoverResult minEdgeCover(const Graph& g,
125                              const std::vector<std::vector<double>>& W,
126                              bool useMST,
127                              const std::vector<MSTEdge>& mstEdges) {
128     const int n = g.n;
129
130     // Build undirected adjacency matrix for the chosen graph
131     std::vector<std::vector<int>>    adjM(n, std::vector<int>(n, 0));
132     std::vector<std::vector<double>> wM(n, std::vector<double>(n, 0.0));
133
134     if (useMST) {
135         for (auto& e : mstEdges) {
136             adjM[e.u][e.v] = adjM[e.v][e.u] = 1;
137             wM[e.u][e.v]   = wM[e.v][e.u]   = e.weight;
138         }
139     } else {
140         for (int i = 0; i < n; i++)
141             for (int j = i + 1; j < n; j++) {
142                 bool edge = g.hasEdge(i, j) || g.hasEdge(j, i);
143                 if (!edge) continue;
144                 adjM[i][j] = adjM[j][i] = 1;
145                 double w = W[i][j] < SHIMBELL_INF / 2 ? W[i][j]
146                          : (W[j][i] < SHIMBELL_INF / 2 ? W[j][i] : 0.0);
147                 wM[i][j] = wM[j][i] = w;
148             }
149     }
150
151     // Find maximum matching
152     std::vector<int> match;
153     int ms = maxMatching(adjM, n, match);
154
155     // Build edge cover: start with matching edges
156     std::vector<bool> covered(n, false);
157     std::vector<MSTEdge> cover;
158
159     for (int u = 0; u < n; u++) {
160         if (match[u] != -1 && u < match[u]) {
161             cover.push_back({ u, match[u], wM[u][match[u]] });
162             covered[u] = covered[match[u]] = true;
163         }
164     }
165
166     // For each unmatched vertex, add any incident edge
167     for (int u = 0; u < n; u++) {
168         if (covered[u]) continue;
169         bool found = false;
170         for (int v = 0; v < n && !found; v++) {
171             if (adjM[u][v]) {
172                 cover.push_back({ u, v, wM[u][v] });
173                 covered[u] = true;
174                 found = true;
175             }
176         }
177         if (!found)
```

```
            std::cout << "  Warning: vertex " << u + 1
                      << " is isolated - cannot be covered.\n";
    }

    return { cover, ms };
}

void printEdgeCoverResult(const EdgeCoverResult& res) {
    const std::string sep(60, '-');
    std::cout << "\n  " << sep << "\n";
    std::cout << "  MINIMUM EDGE COVER\n";
    std::cout << "  " << sep << "\n";
    std::cout << "\n  Maximum matching  |M| = " << res.matchingSize << "\n";
    std::cout << "  Min edge cover   |C| = " << res.cover.size()
              << "  (= n - |M| when no isolated vertices)\n";
    std::cout << "\n  Cover edges:\n";
    std::cout << "    Arc        weight\n";
    std::cout << "    " << std::string(22, '-') << "\n";
    for (auto& e : res.cover)
        std::cout << "    (" << e.u + 1 << " -- " << e.v + 1
                  << ")    " << e.weight << "\n";
    std::cout << "\n  " << sep << "\n";
}
```

Listing 31. File edge_cover.cpp

```
#pragma once
#include "boruvka.h"
#include <vector>

struct PruferResult {
    std::vector<int>    code;     // Prüfer sequence (1-based labels, length n-2)
    std::vector<double> weights;  // weights of n-1 edges in removal order
};

// Encode MST (n vertices, n-1 edges) as Prüfer code with weights.
// Algorithm: repeatedly remove smallest leaf, record its neighbor and edge weight.
PruferResult pruferEncode(const std::vector<MSTEdge>& mstEdges, int n);

// Decode Prüfer code + weights back to a set of edges.
// Returns n-1 edges with assigned weights.
std::vector<MSTEdge> pruferDecode(const std::vector<int>&    code,
                                  const std::vector<double>& weights,
                                  int n);

void printPruferResult(const PruferResult& res, int n);
void printDecodedTree(const std::vector<MSTEdge>& edges);
```

Listing 32. File prufer.h

```
#include "prufer.h"
#include <iostream>
#include <iomanip>
#include <vector>
#include <set>
#include <map>
#include <algorithm>

PruferResult pruferEncode(const std::vector<MSTEdge>& mstEdges, int n) {
    // Build adjacency for the tree: adj[u] = set of (neighbor, weight)
    std::vector<std::map<int, double>> adj(n);
```

```
    for (auto& e : mstEdges) {
        adj[e.u][e.v] = e.weight;
        adj[e.v][e.u] = e.weight;
    }

    // Degree array
    std::vector<int> deg(n, 0);
    for (int i = 0; i < n; i++) deg[i] = (int)adj[i].size();

    // Min-heap of leaves (vertices with degree 1)
    std::set<int> leaves;
    for (int i = 0; i < n; i++)
        if (deg[i] == 1) leaves.insert(i);

    std::vector<int>    code;
    std::vector<double> weights;

    // n-2 iterations produce the Prüfer code
    int remaining = n;
    while (remaining > 2) {
        int leaf = *leaves.begin();
        leaves.erase(leaves.begin());

        // The single neighbor of leaf
        auto it = adj[leaf].begin();
        int   neighbor = it->first;
        double w       = it->second;

        code.push_back(neighbor + 1);   // store 1-based
        weights.push_back(w);

        std::cout << "    Remove leaf " << leaf + 1
                  << "  neighbor=" << neighbor + 1
                  << "  weight=" << w << "\n";

        // Remove leaf from tree
        adj[leaf].clear();
        adj[neighbor].erase(leaf);
        deg[leaf] = 0;
        deg[neighbor]--;
        if (deg[neighbor] == 1) leaves.insert(neighbor);

        remaining--;
    }

    // The last edge (between the two remaining vertices)
    std::vector<int> last;
    for (int i = 0; i < n; i++)
        if (!adj[i].empty()) last.push_back(i);

    if (last.size() == 2) {
        double w = adj[last[0]][last[1]];
        weights.push_back(w);
        std::cout << "    Last edge: (" << last[0] + 1 << ", "
                  << last[1] + 1 << ")  weight=" << w << "\n";
    }

    return { code, weights };
}

```

```cpp
std::vector<MSTEdge> pruferDecode(const std::vector<int>&    code,
                                  const std::vector<double>& weights,
                                  int n) {
    // Degree[v] = 1 + count(v in code), then degree for vertices not in code = 1
    std::vector<int> deg(n, 1);
    for (int x : code) deg[x - 1]++;   // code is 1-based

    // Min-set of leaves (degree 1)
    std::set<int> leaves;
    for (int i = 0; i < n; i++)
        if (deg[i] == 1) leaves.insert(i);

    std::vector<MSTEdge> edges;
    int wi = 0;

    for (int i = 0; i < (int)code.size(); i++) {
        int leaf     = *leaves.begin();
        leaves.erase(leaves.begin());
        int parent   = code[i] - 1;   // convert to 0-based
        double w     = weights[wi++];

        edges.push_back({ leaf, parent, w });
        std::cout << "    Connect " << leaf + 1 << " -- " << parent + 1
                  << "  weight=" << w << "\n";

        deg[leaf]--;
        deg[parent]--;
        if (deg[parent] == 1) leaves.insert(parent);
    }

    // Connect the two remaining vertices with the last stored weight
    std::vector<int> rem;
    for (int i = 0; i < n; i++)
        if (deg[i] == 1) rem.push_back(i);

    if (rem.size() == 2) {
        double w = weights[wi];
        edges.push_back({ rem[0], rem[1], w });
        std::cout << "    Connect " << rem[0] + 1 << " -- " << rem[1] + 1
                  << "  weight=" << w << "  (final edge)\n";
    }

    return edges;
}

void printPruferResult(const PruferResult& res, int n) {
    const std::string sep(60, '-');
    std::cout << "\n  " << sep << "\n";
    std::cout << "  PRUFER CODE  (n=" << n << ", code length=" << res.code.size() << ")\n";
    std::cout << "  " << sep << "\n";
    // std::cout << "\n  Prufer sequence:  [ ";
    // for (int i = 0; i < (int)res.code.size(); i++) {
    //     if (i) std::cout << ", ";
    //     std::cout << res.code[i];
    // }
    // std::cout << " ]\n";
    std::cout <<"\n Prufer sequence: [ ";
    if (res.code.empty())
    {
        std::cout << "0";
```

```
132     } else
133     {
134         for (int i = 0; i < (int)res.code.size(); i++ )
135         {
136             if (i) std::cout << ", ";
137             std::cout << res.code[i];
138         }
139     }
140     std::cout << " ]\n";
141     std::cout << "\n  Edge weights (in removal order, length " << res.weights.size() << "):\
    n  [ ";
142     for (int i = 0; i < (int)res.weights.size(); i++) {
143         if (i) std::cout << ", ";
144         std::cout << res.weights[i];
145     }
146     std::cout << " ]\n";
147     std::cout << "  (last weight is for the final edge not in Prufer sequence)\n";
148     std::cout << "\n  " << sep << "\n";
149 }
150 
151 void printDecodedTree(const std::vector<MSTEdge>& edges) {
152     const std::string sep(60, '-');
153     std::cout << "\n  " << sep << "\n";
154     std::cout << "  DECODED TREE EDGES\n";
155     std::cout << "  " << sep << "\n";
156     double total = 0.0;
157     for (auto& e : edges) {
158         std::cout << "    (" << e.u + 1 << ", " << e.v + 1
159                   << ")  weight=" << e.weight << "\n";
160         total += e.weight;
161     }
162     std::cout << "\n  Total weight = " << total << "\n";
163     std::cout << "\n  " << sep << "\n";
164 }
```

Listing 33. File prufer.cpp

Appendix F. Source Code of Laboratory Work No. 5

```
#pragma once
#include "graph.h"
#include <string>
#include <utility>
#include <vector>

struct EulerianResult {
    bool originalConnected;
    bool originalEulerian;
    bool modificationPossible;
    std::vector<int> originalDegrees;
    std::vector<int> originalOddVertices;
    std::vector<int> modifiedDegrees;
    std::vector<std::pair<int, int>> addedEdges;
    std::vector<std::pair<int, int>> removedEdges;
    std::vector<std::string> log;
    std::vector<int> eulerCycle;
};

// Checks the undirected graph, modifies only edges of a local simple graph
// when needed, and constructs an Euler cycle by Hierholzer's algorithm.
// The input Graph is not mutated.
EulerianResult buildEulerianCycle(const Graph& g);

void printEulerianResult(const EulerianResult& res);
```

Listing 34. File eulerian.h

```
#include "eulerian.h"
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <queue>
#include <sstream>

static inline int D(int v) { return v + 1; }

static int degreeOf(const std::vector<std::vector<int>>& adj, int v) {
    int deg = 0;
    for (int u = 0; u < (int)adj.size(); u++)
        deg += adj[v][u];
    return deg;
}

static std::vector<int> degreesOf(const std::vector<std::vector<int>>& adj) {
    std::vector<int> deg(adj.size(), 0);
    for (int v = 0; v < (int)adj.size(); v++)
        deg[v] = degreeOf(adj, v);
    return deg;
}

static std::vector<int> oddVertices(const std::vector<int>& deg) {
    std::vector<int> odd;
    for (int i = 0; i < (int)deg.size(); i++)
        if (deg[i] % 2 != 0)
            odd.push_back(i);
```

```
    return odd;
}

static bool hasAnyEdge(const std::vector<std::vector<int>>& adj) {
    for (int i = 0; i < (int)adj.size(); i++)
        for (int j = i + 1; j < (int)adj.size(); j++)
            if (adj[i][j] > 0)
                return true;
    return false;
}

static std::vector<int> bfsReachable(const std::vector<std::vector<int>>& adj,
                                     int start) {
    const int n = (int)adj.size();
    std::vector<int> visited(n, 0);
    std::queue<int> q;
    visited[start] = 1;
    q.push(start);

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int u = 0; u < n; u++) {
            if (adj[v][u] > 0 && !visited[u]) {
                visited[u] = 1;
                q.push(u);
            }
        }
    }

    return visited;
}

static std::vector<std::vector<int>> connectedComponents(
        const std::vector<std::vector<int>>& adj) {
    const int n = (int)adj.size();
    std::vector<int> used(n, 0);
    std::vector<std::vector<int>> comps;

    for (int start = 0; start < n; start++) {
        if (used[start]) continue;

        std::vector<int> comp;
        std::queue<int> q;
        q.push(start);
        used[start] = 1;

        while (!q.empty()) {
            int v = q.front();
            q.pop();
            comp.push_back(v);
            for (int u = 0; u < n; u++) {
                if (adj[v][u] > 0 && !used[u]) {
                    used[u] = 1;
                    q.push(u);
                }
            }
        }
        comps.push_back(comp);
    }
```

```
89
90      return comps;
91  }
92
93  static bool isConnected(const std::vector<std::vector<int>>& adj) {
94      const int n = (int)adj.size();
95      if (n <= 1) return true;
96      auto seen = bfsReachable(adj, 0);
97      return std::all_of(seen.begin(), seen.end(), [](int x) { return x != 0; });
98  }
99
100 static void addSimpleEdge(std::vector<std::vector<int>>& adj, int u, int v) {
101     adj[u][v] = 1;
102     adj[v][u] = 1;
103 }
104
105 static void removeSimpleEdge(std::vector<std::vector<int>>& adj, int u, int v) {
106     adj[u][v] = 0;
107     adj[v][u] = 0;
108 }
109
110 static bool canRemoveWithoutDisconnecting(
111         const std::vector<std::vector<int>>& adj, int u, int v) {
112     if (adj[u][v] == 0) return false;
113     auto test = adj;
114     removeSimpleEdge(test, u, v);
115     return isConnected(test);
116 }
117
118 static bool tryEdgeSwapForOddPair(
119         std::vector<std::vector<int>>& adj,
120         int u,
121         int v,
122         EulerianResult& res) {
123     const int n = (int)adj.size();
124
125     for (int x = 0; x < n; x++) {
126         if (x == u || x == v) continue;
127
128         if (adj[u][x] > 0 && adj[v][x] == 0) {
129             addSimpleEdge(adj, v, x);
130             if (canRemoveWithoutDisconnecting(adj, u, x)) {
131                 removeSimpleEdge(adj, u, x);
132                 res.addedEdges.push_back({v, x});
133                 res.removedEdges.push_back({u, x});
134
135                 std::ostringstream add;
136                 add << "Added missing edge (" << D(v) << " -- " << D(x)
137                     << ").";
138                 res.log.push_back(add.str());
139
140                 std::ostringstream rem;
141                 rem << "Removed edge (" << D(u) << " -- " << D(x)
142                     << "); graph remains connected.";
143                 res.log.push_back(rem.str());
144                 return true;
145             }
146             removeSimpleEdge(adj, v, x);
147         }
148
```

```
        if (adj[v][x] > 0 && adj[u][x] == 0) {
            addSimpleEdge(adj, u, x);
            if (canRemoveWithoutDisconnecting(adj, v, x)) {
                removeSimpleEdge(adj, v, x);
                res.addedEdges.push_back({u, x});
                res.removedEdges.push_back({v, x});

                std::ostringstream add;
                add << "Added missing edge (" << D(u) << " -- " << D(x)
                    << ").";
                res.log.push_back(add.str());

                std::ostringstream rem;
                rem << "Removed edge (" << D(v) << " -- " << D(x)
                    << "); graph remains connected.";
                res.log.push_back(rem.str());
                return true;
            }
            removeSimpleEdge(adj, u, x);
        }
    }

    return false;
}

static std::vector<int> hierholzer(std::vector<std::vector<int>> adj) {
    const int n = (int)adj.size();
    int start = 0;
    for (int i = 0; i < n; i++) {
        if (degreeOf(adj, i) > 0) {
            start = i;
            break;
        }
    }

    std::vector<int> stack;
    std::vector<int> cycle;
    stack.push_back(start);

    while (!stack.empty()) {
        int v = stack.back();
        int next = -1;
        for (int u = 0; u < n; u++) {
            if (adj[v][u] > 0) {
                next = u;
                break;
            }
        }

        if (next == -1) {
            cycle.push_back(v);
            stack.pop_back();
        } else {
            adj[v][next] = 0;
            adj[next][v] = 0;
            stack.push_back(next);
        }
    }

    std::reverse(cycle.begin(), cycle.end());
```

```
    return cycle;
}

EulerianResult buildEulerianCycle(const Graph& g) {
    const int n = g.n;
    std::vector<std::vector<int>> adj(n, std::vector<int>(n, 0));

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (g.hasEdge(i, j) || g.hasEdge(j, i))
                addSimpleEdge(adj, i, j);

    EulerianResult res;
    res.originalConnected = isConnected(adj);
    res.originalDegrees = degreesOf(adj);
    res.originalOddVertices = oddVertices(res.originalDegrees);
    res.originalEulerian = res.originalConnected &&
                           res.originalOddVertices.empty() &&
                           hasAnyEdge(adj);
    res.modificationPossible = true;

    if (!hasAnyEdge(adj)) {
        res.log.push_back("Graph has no edges; Euler cycle is not constructed.");
        res.modificationPossible = false;
        res.modifiedDegrees = res.originalDegrees;
        return res;
    }

    if (!res.originalConnected) {
        auto comps = connectedComponents(adj);
        std::ostringstream os;
        os << "Graph is disconnected: " << comps.size()
           << " connected components found.";
        res.log.push_back(os.str());

        for (int i = 0; i + 1 < (int)comps.size(); i++) {
            int u = comps[i][0];
            int v = comps[i + 1][0];
            addSimpleEdge(adj, u, v);
            res.addedEdges.push_back({u, v});
            std::ostringstream add;
            add << "Added edge (" << D(u) << " -- " << D(v)
                << ") to connect components.";
            res.log.push_back(add.str());
        }
    }

    auto currentDegrees = degreesOf(adj);
    auto odd = oddVertices(currentDegrees);
    if (odd.empty()) {
        res.log.push_back("All vertex degrees are even; no parity modification needed.");
    } else {
        const int maxSteps = n * n + 1;
        for (int step = 0; step < maxSteps; step++) {
            currentDegrees = degreesOf(adj);
            odd = oddVertices(currentDegrees);
            if (odd.empty()) break;

            std::ostringstream os;
            os << "Odd vertices:";
```

```
            for (int v : odd) os << " " << D(v);
            res.log.push_back(os.str());

            if (n < 3) {
                res.log.push_back("Cannot modify this graph under restrictions: "
                                  "need at least 3 vertices to avoid duplicate edges "
                                  "and keep the graph connected.");
                res.modificationPossible = false;
                break;
            }

            bool changed = false;

            for (int i = 0; i < (int)odd.size() && !changed; i++) {
                for (int j = i + 1; j < (int)odd.size() && !changed; j++) {
                    int u = odd[i];
                    int v = odd[j];
                    if (adj[u][v] == 0) {
                        addSimpleEdge(adj, u, v);
                        res.addedEdges.push_back({u, v});
                        std::ostringstream add;
                        add << "Added missing edge (" << D(u) << " -- "
                            << D(v) << ").";
                        res.log.push_back(add.str());
                        changed = true;
                    }
                }
            }

            for (int i = 0; i < (int)odd.size() && !changed; i++) {
                for (int j = i + 1; j < (int)odd.size() && !changed; j++) {
                    int u = odd[i];
                    int v = odd[j];
                    if (adj[u][v] > 0 && canRemoveWithoutDisconnecting(adj, u, v)) {
                        removeSimpleEdge(adj, u, v);
                        res.removedEdges.push_back({u, v});
                        std::ostringstream rem;
                        rem << "Removed edge (" << D(u) << " -- "
                            << D(v) << "); graph remains connected.";
                        res.log.push_back(rem.str());
                        changed = true;
                    }
                }
            }

            for (int i = 0; i < (int)odd.size() && !changed; i++) {
                for (int j = i + 1; j < (int)odd.size() && !changed; j++) {
                    int u = odd[i];
                    int v = odd[j];
                    if (adj[u][v] > 0)
                        changed = tryEdgeSwapForOddPair(adj, u, v, res);
                }
            }

            if (!changed) {
                res.log.push_back("Cannot modify this graph under restrictions: "
                                  "no legal add/remove operation was found.");
                res.modificationPossible = false;
                break;
            }
```

```
        }

        currentDegrees = degreesOf(adj);
        odd = oddVertices(currentDegrees);
        if (!odd.empty() && res.modificationPossible) {
            res.log.push_back("Cannot modify this graph under restrictions: "
                              "the add/remove process did not reach all even degrees.");
            res.modificationPossible = false;
        }
    }

    res.modifiedDegrees = degreesOf(adj);
    if (res.modificationPossible && isConnected(adj) &&
        oddVertices(res.modifiedDegrees).empty()) {
        res.eulerCycle = hierholzer(adj);
    } else {
        res.eulerCycle.clear();
    }
    return res;
}

void printEulerianResult(const EulerianResult& res) {
    const std::string sep(60, '-');
    const int n = (int)res.originalDegrees.size();

    std::cout << "\n  " << sep << "\n";
    std::cout << "  LAB 5: EULERIAN GRAPH CHECK AND EULER CYCLE\n";
    std::cout << "  " << sep << "\n\n";

    std::cout << "  Connected: " << (res.originalConnected ? "YES" : "NO") << "\n";
    std::cout << "  Original graph is Eulerian: "
              << (res.originalEulerian ? "YES" : "NO") << "\n\n";

    std::cout << "  Original degrees:\n";
    std::cout << "    Vertex | Degree | Parity\n";
    std::cout << "    " << std::string(26, '-') << "\n";
    for (int i = 0; i < n; i++) {
        std::cout << "      " << std::setw(3) << D(i)
                  << "   |  " << std::setw(3) << res.originalDegrees[i]
                  << "   | " << (res.originalDegrees[i] % 2 ? "odd" : "even")
                  << "\n";
    }

    std::cout << "\n  Modification log:\n";
    for (const auto& line : res.log)
        std::cout << "    - " << line << "\n";

    if (!res.modificationPossible) {
        std::cout << "\n  " << sep << "\n";
        return;
    }

    std::cout << "\n  Modified degrees:\n";
    for (int i = 0; i < (int)res.modifiedDegrees.size(); i++) {
        std::cout << "    deg(" << D(i) << ") = " << res.modifiedDegrees[i]
                  << "  " << (res.modifiedDegrees[i] % 2 ? "odd" : "even")
                  << "\n";
    }

    std::cout << "\n  Euler cycle:\n    ";
```

```
    if (res.eulerCycle.empty()) {
        std::cout << "Not constructed.";
    } else {
        for (int i = 0; i < (int)res.eulerCycle.size(); i++) {
            if (i) std::cout << " -> ";
            std::cout << D(res.eulerCycle[i]);
        }
    }
    std::cout << "\n\n  " << sep << "\n";
}
```

Listing 35. File eulerian.cpp

```
#pragma once
#include "boruvka.h"
#include "graph.h"
#include <vector>

struct CutsetEdge {
    int u, v;
};

struct FundamentalCutset {
    int index;
    MSTEdge treeEdge;
    std::vector<CutsetEdge> edges;
};

std::vector<FundamentalCutset> buildFundamentalCutsets(
        const Graph& g,
        const std::vector<MSTEdge>& mstEdges);

void printFundamentalCutsets(const std::vector<FundamentalCutset>& cutsets);

void printCutsetSymmetricDifference(
        const std::vector<FundamentalCutset>& cutsets,
        const std::vector<int>& selectedIndices);
```

Listing 36. File fundamental_cutsets.h

```
#include "fundamental_cutsets.h"
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <queue>
#include <set>

static inline int D(int v) { return v + 1; }

static std::vector<CutsetEdge> graphEdges(const Graph& g) {
    std::vector<CutsetEdge> edges;
    for (int i = 0; i < g.n; i++)
        for (int j = i + 1; j < g.n; j++)
            if (g.hasEdge(i, j) || g.hasEdge(j, i))
                edges.push_back({i, j});
    return edges;
}

static std::vector<int> treeComponentAfterRemoving(
        int n,
        const std::vector<MSTEdge>& tree,
```

```
        int removedIndex,
        int start) {
    std::vector<std::vector<int>> adj(n);
    for (int i = 0; i < (int)tree.size(); i++) {
        if (i == removedIndex) continue;
        int u = tree[i].u;
        int v = tree[i].v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    std::vector<int> inComponent(n, 0);
    std::queue<int> q;
    q.push(start);
    inComponent[start] = 1;

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int u : adj[v]) {
            if (!inComponent[u]) {
                inComponent[u] = 1;
                q.push(u);
            }
        }
    }

    return inComponent;
}

std::vector<FundamentalCutset> buildFundamentalCutsets(
        const Graph& g,
        const std::vector<MSTEdge>& mstEdges) {
    std::vector<FundamentalCutset> result;
    auto edges = graphEdges(g);

    for (int i = 0; i < (int)mstEdges.size(); i++) {
        const auto& treeEdge = mstEdges[i];
        auto left = treeComponentAfterRemoving(g.n, mstEdges, i, treeEdge.u);

        FundamentalCutset cut;
        cut.index = i + 1;
        cut.treeEdge = treeEdge;

        for (auto e : edges) {
            if (left[e.u] != left[e.v])
                cut.edges.push_back(e);
        }

        std::sort(cut.edges.begin(), cut.edges.end(),
                  [](const CutsetEdge& a, const CutsetEdge& b) {
                      if (a.u != b.u) return a.u < b.u;
                      return a.v < b.v;
                  });

        result.push_back(cut);
    }

    return result;
}
```

```
82
83  void printFundamentalCutsets(const std::vector<FundamentalCutset>& cutsets) {
84      const std::string sep(60, '-');
85      std::cout << "\n  " << sep << "\n";
86      std::cout << "  LAB 5: FUNDAMENTAL CUTSET SYSTEM (EVEN VARIANT)\n";
87      std::cout << "  " << sep << "\n\n";
88
89      if (cutsets.empty()) {
90          std::cout << "  No fundamental cutsets exist.\n";
91          std::cout << "  Reason: a fundamental cutset is built for each MST edge,\n";
92          std::cout << "  and this graph has no MST edges.\n";
93          std::cout << "\n  " << sep << "\n";
94          return;
95      }
96
97      for (const auto& cut : cutsets) {
98          std::cout << "  Cutset " << cut.index
99                    << "  for tree edge (" << D(cut.treeEdge.u)
100                   << " -- " << D(cut.treeEdge.v) << "):\n";
101         std::cout << "    { ";
102         for (int i = 0; i < (int)cut.edges.size(); i++) {
103             if (i) std::cout << ", ";
104             std::cout << "(" << D(cut.edges[i].u)
105                       << "--" << D(cut.edges[i].v) << ")";
106         }
107         std::cout << " }\n";
108     }
109
110     std::cout << "\n  " << sep << "\n";
111 }
112
113 void printCutsetSymmetricDifference(
114         const std::vector<FundamentalCutset>& cutsets,
115         const std::vector<int>& selectedIndices) {
116     std::set<std::pair<int, int>> result;
117
118     for (int idx : selectedIndices) {
119         if (idx < 1 || idx > (int)cutsets.size()) continue;
120         for (auto e : cutsets[idx - 1].edges) {
121             auto key = std::make_pair(e.u, e.v);
122             auto it = result.find(key);
123             if (it == result.end())
124                 result.insert(key);
125             else
126                 result.erase(it);
127         }
128     }
129
130     std::cout << "\n  Symmetric difference of selected cutsets";
131     if (!selectedIndices.empty()) {
132         std::cout << " (";
133         for (int i = 0; i < (int)selectedIndices.size(); i++) {
134             if (i) std::cout << ", ";
135             std::cout << selectedIndices[i];
136         }
137         std::cout << ")";
138     }
139     std::cout << ":\n";
140
141     std::cout << "    { ";
```

```
    int printed = 0;
    for (auto [u, v] : result) {
        if (printed++) std::cout << ", ";
        std::cout << "(" << D(u) << "--" << D(v) << ")";
    }
    std::cout << " }\n";
}
```

Listing 37. File fundamental_cutsets.cpp